Tecnologías Extrañas: Cifrado Homomórfico

Apuestas asimétricas para ingenieros curiosos

Marc Magrans de Abril

Tabla de contenidos

Prefacio **1**

Prólogo **3**

Agradecimientos **5**

Descargo de responsabilidad **7**

1 Introducción **9**
1.1 ¿Por qué esta tecnología? 9
1.2 Breve historia 10

2 Manos a la obra **13**
2.1 Receta 1: Media cifrada 15
2.2 Receta 2: Varianza cifrada 16
2.3 Receta 3: Regresión lineal cifrada 18
2.4 Receta 4: El muro del ruido 19
2.5 Receta 5: Rompiendo el muro 21
2.6 Receta 6: Búsqueda cifrada 24

3 Aplicaciones **27**

4 Limitaciones actuales **29**

5 Fundamentos **33**
5.1 Retículos 34
5.2 De los retículos a los anillos de polinomios: LWE y RLWE . 35
5.3 Cifrado y operaciones homomórficas 40
5.4 Ruido y niveles 46
5.5 Bootstrapping 53

5.6 Packing y SIMD 57
5.7 Rotaciones 62
5.8 Otros esquemas de encriptado 69

Lecturas Recomendadas **73**

Referencias **75**

Soluciones **77**

Prefacio

Este libro forma parte de la serie **Tecnologías Extrañas**: pequeños libros independientes sobre tecnologías avanzadas que la mayoría de ingenieros no conoce, pero que podrían necesitar en el futuro y que representan una oportunidad profesional.

Una Tecnología Extraña se caracteriza, en primer lugar, por haber sobrevivido al paso del tiempo, superando así el filtro de Lindy descubierto por B. Mandelbrot y luego popularizado por N. Taleb. Las bicicletas, las ventanas, el buen vino y los buenos amigos poseen esta cualidad. Cuanto más tiempo viven, más seguros estamos de que sobrevivirán al paso del tiempo — su esperanza de vida condicionada es proporcional a su edad actual. Igualmente, estas tecnologías llevan tiempo desarrollándose y utilizándose en ciertos ámbitos, y por lo tanto, es probable que sigan en escena en un futuro.

En segundo lugar, y como consecuencia de haber superado el filtro de Lindy, una Tecnología Extraña tiene unos fundamentos matemáticos robustos que la establecen como apuesta de futuro, y por lo tanto, hay un flujo continuo o creciente de artículos científicos, noticias y software que expanden esos fundamentos y sus aplicaciones.

En tercer y último lugar, una Tecnología Extraña es una apuesta de futuro en el sentido de que no ha sido totalmente explotada por el mercado, y por lo tanto, el núcleo del campo puede aprenderse con relativamente poco esfuerzo y puede contener nichos aún inexplorados.

Prólogo

Elena, directora de innovación de una aseguradora médica, entra en la sala con una carpeta llena de post-its. Siempre impecable y con la confianza de una carrera brillante, hoy parece estar un tanto descolocada. Lleva varias semanas intentando cerrar un acuerdo con un proveedor de analítica en la nube. Daniel, ingeniero principal del equipo de datos, la espera con el portátil abierto después de haberse tomado un café y haber leído las últimas noticias de Hacker News. Al menos ha empezado bien el día, pero parece que se acercan las malas noticias. ¿No es curioso que en este mundo caótico, haya situaciones tan predecibles?

Elena: *Daniel, necesito que el proveedor analice los datos de nuestros pacientes para los modelos de riesgo. Pero el departamento legal nos ha bloqueado. Dicen que los datos médicos son confidenciales y no podemos enviarlos a un tercero. ¿No hay forma de que hagan el análisis sin ver los datos?*

Daniel: *Operar sobre datos sin verlos… eso suena a pedirle al chef que cocine sin tocar los ingredientes.*

Elena: *No me vengas con metáforas. Necesito una solución real. El contrato se firma en tres semanas o perdemos al proveedor.*

Daniel: *Un momento, déjame pensar… «cocinar sin ver los ingredientes»… Esto me recuerda que hace un par de semanas, mientras tomaba café con un colega me habló de una nueva tecnología, una idea descabellada casi de ciencia ficción. Existe algo llamado cifrado homomórfico que permite enviar los datos cifrados a la nube, y allí se ejecuta el análisis sobre los datos cifrados, pero sin descifrar. Uno pensaría que eso no tendría que funcionar, pero resulta que se pueden hacer los cálculos sobre la información cifrada sin necesidad de descifrarla. El resultado también está cifrado, y solo nosotros lo podemos descifrar. Ellos nunca ven un solo nombre ni un solo diagnóstico. Es como ser capaz de cocinar sin ver los ingredientes.*

Elena: *¿Y eso funciona? ¿Por qué no lo usa todo el mundo?*

Daniel: *Porque hasta hace poco era impracticable —demasiado lento. Pero la tecnología ha madurado lo suficiente para ciertos casos. El nuestro podría ser uno. ¿Quieres que me lo mire?*

Elena: *Sí, claro. ¿Me haces una propuesta para mañana?*

Daniel mira a Elena con cara de desconcierto. «Para mañana.» Dos palabras que en gestión de proyectos significan «no he entendido absolutamente nada de lo que me acabas de decir, pero suena a que debería ser fácil». Le tendría que haber dicho que no estamos en Star Trek, y claramente, yo no soy Scotty. ¿En qué momento la palabra «nueva tecnología» no le ha quedado registrada?

Agradecimientos

Este libro no se ha creado de la nada. Se ha creado gracias a la acumulación de conocimiento y riqueza pasados, y a la ayuda incondicional de la familia, amigos, y colegas. Respecto al pasado, he hecho un pésimo trabajo al no poder dar el debido reconocimiento a todas las personas e instituciones que han hecho posible dicha acumulación y de las que el libro se ha beneficiado, pero en aras de la brevedad, he decidido referenciar solo los documentos esenciales para entender y profundizar en este campo. Respecto a las personas, no puedo ni imaginar cómo podría haber escrito una sola palabra sin la educación de mis padres, Ildefons y Pili, el amor de Bego, mis hermanos, y mis hijos, o sin las risas y consejos de mis amigos. Cualquier mérito debe asociarse a toda esta alud de personas más y menos anónimas que han contribuido. Muchas gracias por toda vuestra ayuda, amor, y amistad.

Descargo de responsabilidad

Las opiniones y puntos de vista expresados en este libro son únicamente míos y no reflejan necesariamente los de mis empleadores, colegas o amigos, pasados o presentes. He hecho todo lo posible por documentarme, contrastar los distintos puntos de vista y estudiar la literatura relevante, apoyándome también en asistentes de inteligencia artificial como herramienta de investigación, redacción y revisión. Aun así, he verificado personalmente cada afirmación, cifra, table, figura, fragmento de código o formulación matemática antes de incluirlo. Cualquier error, inexactitud o malinterpretación que persista es responsabilidad exclusivamente mía, y pido disculpas de antemano por ella.

Sobre todo, lo más importante y fundamental, es que no utilicéis nada de este libro para lanzar una nave a Marte, darle el toque final al teletransportador cuántico que estáis acabando en vuestro garaje, o convencer a vuestro hijo adolescente de que apague el móvil. No va a funcionar. Este libro no tiene la profundidad de contexto necesaria para tales fines. Es solo una introduccion al cifrado homorfico y sus aplicaciones y no pretende ser otra cosa.

Capítulo 1

Introducción

El cifrado homomórfico permite realizar computaciones sobre datos cifrados sin necesidad de descifrarlos. Esto abre la puerta a un paradigma donde la privacidad y la computación en la nube dejan de ser conceptos opuestos. El precio: las operaciones homomórficas son todavía entre 10^4 y 10^6 veces más lentas que en claro, aunque hardware especializado y compiladores dedicados pueden recortar esa brecha a 10–100× (Capítulo 4), lo que limita su uso a escenarios donde la privacidad justifica el coste.

1.1 ¿Por qué esta tecnología?

El cifrado homomórfico cumple todos los requisitos de una Tecnología Extraña. Tiene, como veremos más adelante, unos fundamentos matemáticos robustos y con una larga historia, tiene una atención considerable en términos de publicaciones y noticias sin ser parte del mainstream, y no es una tecnología comúnmente conocida. Además, la idea de «computar sin ver los datos» es en sí fascinante, y atrae tanto a ingenieros como inversores.

Cualitativamente, es una gran elección como Tecnología Extraña: (1) tiene tracción empírica real — Microsoft SEAL, Zama Concrete y las librerías de IBM están disponibles; (2) es *extraña* incluso para criptógrafos experimentados — la idea de operar sobre datos cifrados desafía la intuición; (3) supera el filtro de Lindy con una base teórica y práctica sólida desarrollada durante un largo periodo de tiempo — Rivest planteó la pregunta en 1978, Gentry la resolvió en 2009, y los retículos que la sustentan tienen raíces en la geometría de los números del siglo XIX; (4) su núcleo matemático — polinomios en anillos cociente y ruido controlado — se pueden aprender

los rudimentos en un tiempo razonable, como se verá en este capítulo; (5) comparte la base teórica de retículos con la criptografía post-cuántica, lo que la hace doblemente interesante; y (6) no es la apuesta principal del mercado — el mainstream habla de computación confidencial con enclaves hardware o privacidad diferencial, no de esquemas homomórficos.

1.2 Breve historia

El nombre *homomórfico* viene del griego *homo* (mismo) y *morphḗ* (forma): un cifrado que preserva la forma de las operaciones, de modo que lo que se hace con los datos cifrados produce el mismo resultado que si se hiciera con los datos en claro. La mejor forma de entenderlo es con una analogía (Gentry 2010): imagina que Alice, una joyera muy desconfiada, mete sus materiales en bruto dentro de una caja de guantes transparente y la cierra con llave. Envía la caja a un trabajador que puede manipular las piezas a través de los guantes —cortarlas, soldarlas, pulirlas— pero sin tocarlas jamás directamente. Cuando termina, Alice recupera la caja y la abre con su llave: dentro encuentra el anillo terminado. El trabajador ha hecho todo el trabajo sin haber tenido acceso a los materiales en ningún momento. La pregunta es si semejante cosa es posible con datos y sus computaciones asociadas.

En 1978, Rivest, Adleman y Dertouzos plantearon exactamente esa pregunta matemáticamente: ¿se pueden realizar cálculos sobre datos cifrados sin descifrarlos (Rivest, Adleman, y Dertouzos 1978)? Durante tres décadas la respuesta práctica fue «no». Los esquemas existentes —RSA, ElGamal, Paillier— soportaban *una* operación (multiplicación o suma), pero no ambas; se les llama *parcialmente* homomórficos.

En 2009, Craig Gentry resolvió el problema abierto en su tesis doctoral, construyendo el primer esquema *completamente* homomórfico (*Fully Homomorphic Encryption*, FHE) sobre retículos ideales (Gentry 2009). La clave de su construcción es el *bootstrapping*: un esquema «algo homomórfico» evalúa su propio circuito de descifrado, lo que refresca el ruido del cifrotexto — las imprecisiones que se van acumulando con cada operación — y permite operaciones ilimitadas.

Desde entonces, sucesivas generaciones de esquemas —BGV (2012), BFV (2012), GSW (2013), CKKS (2017), TFHE (2018)— han reducido el coste computacional en varios órdenes de magnitud, hasta hacer viables los primeros despliegues reales.

En los siguientes capítulos veremos primero cómo se puede operar con ci-

frado homomórfico en la práctica, después las posibles aplicaciones y limitaciones de la tecnología (Capítulo 4), y para concluir, los fundamentos matemáticos de dichas operaciones (Capítulo 5), todo ello intercalado con preguntas de varios niveles para cerciorarnos de que vamos entendiendo los conceptos.

Capítulo 2

Manos a la obra

Ni Daniel, ni tampoco el mejor de los ingenieros, sabe por dónde empezar cuando experimenta con una tecnología nueva. Una cosa es hacer un prototipo, y otra muy distinta es construir un sistema en producción con un nivel de servicio adecuado a la aplicación. Pero una forma de familiarizarse con las intuiciones básicas de una tecnología es empezar a experimentar con ella. Intentar entender la teoría matemática subyacente sin una intuición sobre su funcionamiento suele ser casi imposible.

O sea que antes de sumergirnos en la teoría, veamos el FHE en acción con seis simples recetas: empezaremos calculando una media sobre datos cifrados, pasaremos a la varianza — que ya exige multiplicaciones —, intentaremos una regresión lineal, chocaremos contra el muro del ruido, lo romperemos con *bootstrapping*, y cerraremos con una búsqueda cifrada donde el servidor responde sin saber qué le preguntaron. Cada receta plantea un pequeño problema y lo resuelve paso a paso. Al final de la sección sabrás qué puede y qué no puede hacer el FHE *hoy*, antes de entender el porqué.

Todas las recetas usan Python con TenSEAL, una librería que implementa el esquema CKKS — el más adecuado para operaciones con números reales —, salvo la Receta 5, que recurre a OpenFHE porque TenSEAL no expone *bootstrapping*.

El primer paso, común a todas las recetas, es crear el *contexto criptográfico* — el equivalente a preparar la «caja de guantes» de la analogía. Cada parámetro usado en la inicialización de la librería tiene un porqué que veremos en la Capítulo 5:

```python
import tenseal as ts

contexto = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 40, 60],
)
contexto.global_scale = 2**40
contexto.generate_galois_keys()
```

De momento basta saber esto:

- `CKKS` es el esquema para operar con números reales aproximados. Otros esquemas trabajan con enteros o bits — ver Tabla 5.3.
- `poly_modulus_degree = 8192` fija el grado N del polinomio que subyace al esquema. N no se elige en función de cuántos datos tengamos, sino de los requisitos de seguridad, la profundidad del circuito (cuántas multiplicaciones encadenadas necesitamos) y la capacidad de paralelización deseada. Un N mayor da más seguridad y más capacidad de paralelización — cada cifrotexto puede contener hasta $N_{\text{slot}} = N/2$ valores independientes, llamados *slots* —, pero cada operación es proporcionalmente más lenta. Con $N = 8192$ tenemos $N_{\text{slot}} = 4096$ slots disponibles, aunque nuestros ejemplos usan muchos menos.
- `coeff_mod_bit_sizes = [60, 40, 40, 60]` determina cuántas multiplicaciones encadenadas se pueden hacer antes de que el ruido acumulado destruya el resultado. Aquí, los dos valores intermedios de 40 bits dan dos niveles de multiplicación — ver Sección 5.4.
- `global_scale = 2**40` fija la precisión con que se representan los números reales dentro del cifrotexto. A mayor escala, menor ruido de aproximación — pero menos margen queda para multiplicaciones encadenadas. Con 2^{40}, el error absoluto de una operación sencilla como la media es del orden de 10^{-7}, independientemente de si los operandos son miles o decimales.
- `generate_galois_keys()` genera las claves necesarias para *rotaciones* — operaciones que combinan valores de distintos *slots* del cifrotexto. Sin ellas, `sum()` fallaría. Se generan explícitamente porque ocupan cientos de megabytes — ver Sección 5.6.

Con este contexto listo, veamos la primera receta.

2.1 Receta 1: Media cifrada

Daniel decide empezar por lo más sencillo: calcular la media salarial sobre datos cifrados y comprobar que el resultado coincide con el cálculo en claro. Ciframos los datos y, a partir de este momento, las operaciones se ejecutan sobre el cifrotexto — los datos en claro ya no intervienen:

```
salarios = [3200, 4100, 2800, 5500, 3900]
salarios_cifrados = ts.ckks_vector(contexto, salarios)

# En un despliegue real, esto se ejecutaría en la nube,
# sin acceso a los datos en claro
# (CKKSVector no implementa la división `/`,
# solo la multiplicación `*`, así que dividir por n se expresa
# como una multiplicación por la constante 1/n)
media_cifrada = salarios_cifrados.sum() * (1 / len(salarios))
```

Los cinco salarios se empaquetan en un solo cifrotexto gracias al *packing* (Sección 5.6), que con nuestra configuración podría empaquetar hasta $N/2 = N_{\text{slot}} = 4096$ valores. La operación `sum()` usa rotaciones internas para sumar todos los *slots*, y la división por 5 es una multiplicación por una constante (1/5), barata en FHE.

Finalmente, desciframos el resultado:

```
media = media_cifrada.decrypt()[0]
media_claro = sum(salarios) / len(salarios)
print(f"Media (FHE):   {media:.2f} €")  # 3900.00 €
print(f"Media (claro): {media_claro:.2f} €")  # 3900.00 €
```

El resultado es prácticamente idéntico — la diferencia absoluta es del orden de 10^{-7}, un ruido inherente al esquema CKKS que permanece constante independientemente de la magnitud de los operandos (veremos por qué en Sección 5.4), y el tiempo de ejecución. Nuestro ejemplo se ejecuta en local, pero las operaciones sobre el cifrotexto serían idénticas en un despliegue real: el proveedor en la nube habría calculado la media salarial sin haber visto un solo número real. Exactamente lo que Elena necesitaba.

Los parámetros del contexto parecen arbitrarios a primera vista — ¿por qué 8192 y no 4096? ¿Por qué `[60, 40, 40, 60]`? ¿Por qué 2^{40}? Cada decisión tiene una razón matemática precisa que veremos en la Capítulo 5. Pero antes, las siguientes recetas muestran lo que FHE permite — y lo que no.

Preguntas de repaso

2.1. Pídele a un LLM: *«¿Qué diferencia hay entre los esquemas BFV y CKKS? ¿Cuándo usarías cada uno?»*

2.2. ¿Qué ocurre si aumentas `poly_modulus_degree` a 16384? ¿Y si lo reduces a 4096? Experimenta con el tiempo de ejecución.

2.2 Receta 2: Varianza cifrada

Daniel envía la media a Elena, pero ella quiere más: *«Necesito la dispersión — ¿los salarios son homogéneos o hay outliers?»* La varianza poblacional es $\text{Var} = E[X^2] - (E[X])^2$, y calcularla sobre datos cifrados nos obliga a usar la operación más cara del FHE: la multiplicación entre cifrotextos.

Hay un detalle sutil que merece atención. La traducción *ingenua* de la fórmula sería:

```
#  Esto NO funciona correctamente en CKKS
ct = ts.ckks_vector(contexto, salarios)

cuadrados = ct * ct                          # cifrado × cifrado
media_cuadrados = cuadrados.sum() * (1/n)    # × constante

media = ct.sum() * (1/n)                     # × constante
media_al_cuadrado = media * media            # cifrado × cifrado

varianza = media_cuadrados - media_al_cuadrado  # → basura
```

El código es matemáticamente correcto, pero produce un resultado completamente equivocado (en nuestro ejemplo, una varianza *negativa*). ¿Por qué? Cada multiplicación — entre cifrotextos o por una constante — va seguida de un *rescaling* que consume un eslabón de 40 bits de `coeff_mod_bit_sizes` (Sección 5.4 explicará por qué). Con `[60, 40, 40, 60]` solo hay dos eslabones de ese tipo — dos multiplicaciones de margen — antes de llegar al último eslabón de la cadena: el primer valor, de 60 bits, el único que sobrevive cuando se agotan los demás.

Veamos a qué eslabón llega cada rama en este ejemplo:

Rama $E[X^2]$: `ct * ct` (consume 1 eslabón) → `.sum()` (gratis) → `* (1/n)` (consume 1 eslabón) → llega al último eslabón.

Rama $(E[X])^2$: `ct.sum()` (gratis) → `* (1/n)` (consume 1 eslabón) → `media * media` (consume 1 eslabón) → llega también al último eslabón.

Ambas ramas consumen los dos eslabones disponibles y llegan al mismo punto de la cadena — de hecho, instrumentando el código se comprueba que terminan con la *misma* escala y los *mismos* parámetros internos (`scale` y `parms_id` idénticos): no hay ningún desajuste entre ramas. El problema es otro, y más sutil. Al consumir dos eslabones, solo queda el primer módulo de la cadena (60 bits) y, con una escala $\Delta = 2^{40}$, eso deja apenas $60 - 40 = 20$ bits — algo más de un millón — para representar la parte entera del resultado. Pero tanto $E[X^2]$ como $(E[X])^2$ rondan aquí los 15-16 millones, muy por encima de ese margen: el valor se desborda en silencio módulo ese último eslabón, y ninguna de las dos ramas da el resultado correcto, aunque lleguen con metadatos idénticos.

La solución no es reordenar las operaciones — reordenarlas no cambia cuántos eslabones se consumen, y el desbordamiento persiste igual —, sino no agotar nunca ese último eslabón. Si eliminamos la multiplicación por constante, moviendo la división por n al texto claro *antes* de cifrar, cada rama pasa de dos multiplicaciones (2 eslabones) a una sola (1 eslabón), dejando siempre de sobra el eslabón de 40 bits además del primero:

```
ct = ts.ckks_vector(contexto, salarios)
norm = [x / len(salarios) for x in salarios]
ct_n = ts.ckks_vector(contexto, norm)

# E[X²] = sum(x · x/n): una mult cifrado×cifrado (nivel 1)
media_cuadrados = (ct * ct_n).sum()

# (E[X])² = (sum(x/n))²: una mult cifrado×cifrado (nivel 1)
media = ct_n.sum()
media_al_cuadrado = media * media

# Ambos al mismo nivel y escala - la resta funciona
varianza_cifrada = media_cuadrados - media_al_cuadrado
print(f"Varianza (FHE): {varianza_cifrada.decrypt()[0]:.2f}")
```

Confirmemos el resultado, rama por rama:

Rama $E[X^2]$: `ct * ct_n` (única multiplicación, entre dos cifrotextos frescos) → `.sum()` (gratis) → se detiene un eslabón antes del fondo de la cadena.

Rama $(E[X])^2$: `ct_n.sum()` (gratis) → `media * media` (única multiplicación) → se detiene también un eslabón antes del fondo.

Cada rama consume ahora 1 eslabón en lugar de 2, deteniéndose un escalón por encima del fondo de la cadena: en vez de quedarnos solo con el primer

módulo de 60 bits, conservamos también un eslabón de 40 bits adicional, lo que multiplica el margen disponible por un factor de 2^{40} — de sobra para representar unos pocos millones sin desbordarse. Por eso la resta final da el resultado correcto.

Este es un ejemplo real de lo que hace la programación FHE difícil: no basta con traducir una fórmula matemática correcta a código. Cada multiplicación — entre cifrotextos o por una constante — consume parte del presupuesto de niveles, y agotarlo del todo puede dejar el resultado sin margen de precisión suficiente, incluso cuando las dos ramas llegan con parámetros internos idénticos. El programador debe diseñar el circuito pensando no solo en *qué* calcula, sino también en cuánto margen de precisión le queda al resultado final en el peor caso — y dejar siempre un eslabón de más.

Preguntas de repaso

2.3. En la Receta 2, ¿por qué la fórmula $E[X^2] - (E[X])^2$ requiere dos multiplicaciones entre cifrotextos pero la fórmula alternativa $\frac{1}{n}\sum(x_i - \bar{x})^2$ podría requerir más? ¿Cuál es más eficiente en FHE?

2.4. Elena vuelve con un nuevo encargo: en vez de la varianza de un único vector, necesita la **covarianza** entre dos vectores cifrados privados X e Y: $\mathrm{Cov}(X, Y) = E[XY] - E[X]E[Y]$. ¿Cuántos eslabones consumiría la implementación?

2.3 Receta 3: Regresión lineal cifrada

Elena, como siempre, no para y ahora necesita algo más ambicioso: un modelo predictivo que asigne una puntuación de riesgo a cada paciente. *«¿Lo tienes para esta tarde? Es súper urgente y estamos con el agua al cuello. El proveedor tiene un modelo de regresión lineal ya entrenado (los pesos son públicos), pero los datos del paciente son confidenciales.»*

Afortunadamente, FHE permite exactamente esto: cifrar los datos del paciente, multiplicarlos por los pesos en claro y sumar — un producto escalar homomórfico.

```
# Datos del paciente (cifrados) - el proveedor no los ve
features = [1.2, 0.7, 3.1, 0.4, 2.8]
features_cifrados = ts.ckks_vector(contexto, features)

# Pesos del modelo (en claro) - no son confidenciales
```

```
pesos = [0.5, -1.2, 0.8, 0.3, -0.6]

# Predicción: producto escalar homomórfico
prediccion = (features_cifrados * pesos).sum()
print(f"Riesgo (FHE): {prediccion.decrypt()[0]:.4f}")
```

Nótese que aquí no se consume ningún nivel multiplicativo: `features_cifrados * pesos` es una multiplicación por constantes no cifradas (barata), y `sum()` solo usa rotaciones para hacer el calculo eficiente con las claves de Galois. El mismo contexto que ya creamos serviría para un modelo con miles de datos por paciente — gracias al *packing* (Sección 5.6), los $N_{\text{slot}} = 4096$ slots del cifrotexto se procesarían todos en paralelo. Esta es la base de los despliegues reales de *Machine Learning sobre datos cifrados* que comentaremos en la sección de aplicaciones (Capítulo 3).

Preguntas de repaso

2.5. En la receta de regresión lineal, ¿por qué la multiplicación `features_cifrados * pesos` no consume niveles multiplicativos? ¿Qué pasaría si los pesos también estuvieran cifrados?

2.6. Si el modelo tuviera 10 000 features y $N_{\text{slot}} = 4096$, ¿cuántos cifrotextos necesitarías para una predicción? ¿Cómo combinarías los resultados parciales?

2.4 Receta 4: El muro del ruido

Después de una semana infernal, Daniel está cerrando el ordenador de su oficina y recibe una llamada de Elena desesperada a las cinco de la tarde — siempre a las 5 o a las 6, especialmente los viernes. *«¡Daniel! ¡No está funcionando! ¡El círculo de la muerte otra vez! ¿Y además dice que los números se "desbordan"? ¡¿Los números se desbordan?! ¿Y adónde van? ¡Qué cara han puesto cuando les he enseñado la versión que me dijiste que funcionaba! ¿No lo testeaste?».*

«Sí, por supuesto, antes de instalarlo siempre lo testeamos.»

«Pues no sé qué puede haber salido mal. Como no estabas durante la hora de comer, le pedí a Gabriel que añadiera un factor de corrección adicional.»

«¿Es que quizás añadió una multiplicación al resultado final?».

«Pero ¿y eso qué más dará? No sé cómo ha metido la pata…»

¿Qué ocurre si pedimos más operaciones de las que la cadena de módulos permite? Creemos un contexto con un solo nivel multiplicativo e intentemos dos multiplicaciones encadenadas:

```
ctx_1nivel = ts.context(
    ts.SCHEME_TYPE.CKKS,
    poly_modulus_degree=8192,
    coeff_mod_bit_sizes=[60, 40, 60],  # solo 1 nivel
)
ctx_1nivel.global_scale = 2**40
ctx_1nivel.generate_galois_keys()

datos = ts.ckks_vector(ctx_1nivel, [3.0, 7.0, 2.0])

resultado = datos * datos   # OK - consume el único nivel
print(resultado.decrypt())  # [9.0, 49.0, 4.0]

resultado = resultado * resultado  # FALLA - no quedan niveles
# → Error: scale out of bounds
```

La librería lanza una excepción en la segunda multiplicación: se ha pedido un tercer eslabón cuando solo había presupuesto para dos, y TenSEAL se niega a continuar en vez de devolver un resultado sin sentido. Es una decisión de diseño deliberada — pero, como vimos en la Receta 2, no es una garantía universal: ese error explícito solo aparece cuando se pide *más* presupuesto del disponible. Si en cambio el cálculo se queda exactamente en el último eslabón, con el margen de precisión justo al límite —como le pasó a Daniel con la varianza—, CKKS no detecta nada raro y puede devolver basura con toda la apariencia de un resultado válido. La lección práctica es la misma en ambos casos: no apurar nunca el presupuesto de niveles hasta el límite.

Este muro es la manifestación práctica de todo lo que veremos en Sección 5.4: cada nivel de 40 bits en `coeff_mod_bit_sizes` es un eslabón que se consume con cada multiplicación. Cuando se agotan, no hay más camino — a menos que se recurra al *bootstrapping*, que restaura los niveles a un coste enorme (véase la Sección 2.5 para un ejemplo práctico y la Capítulo 5 para la explicación teórica).

Preguntas de repaso

2.7. Si necesitas calcular un polinomio de grado 3 sobre datos cifrados ($ax^3 + bx^2 + cx + d$), ¿cuántos niveles multiplicativos necesitas? ¿Qué `coeff_mod_bit_sizes` configurarías?

2.8. ¿Por qué la librería lanza un error en vez de devolver un resultado aproximado? ¿Qué consecuencias tendría si produjera basura silenciosa en un sistema médico o financiero?

2.5 Receta 5: Rompiendo el muro

La Receta 4 terminó con un error — nos quedamos sin niveles multiplicativos. Pero el *bootstrapping* permite restaurarlos y seguir operando. TenSEAL no expone esta funcionalidad, así que para esta receta usamos OpenFHE, otra librería de referencia que sí implementa bootstrapping en CKKS.

El primer paso es configurar un contexto CKKS con bootstrapping habilitado. Esto requiere niveles extra para evaluar su propio circuito interno. Para que la demo sea rápida, usamos un anillo pequeño ($N = 4096$) con la validación de seguridad desactivada — con `HEStd_128_classic` (128 bits de seguridad frente a ordenadores clásicos, el estándar habitual), OpenFHE exigiría $N = 2^{16}$ para soportar la profundidad del bootstrapping. Existen niveles más exigentes (192, 256 bits) y variantes cuánticas que asumen ataques con ordenadores cuánticos y requieren N aún mayores:

```
import openfhe as fhe

params = fhe.CCParamsCKKSRNS()
params.SetSecretKeyDist(fhe.SecretKeyDist.UNIFORM_TERNARY)
# Anillo pequeño para demo rápida; en producción usar
# HEStd_128_classic con un N mayor.
params.SetSecurityLevel(fhe.SecurityLevel.HEStd_NotSet)
params.SetRingDim(1 << 12)

# El bootstrapping consume niveles propios para evaluar
# el circuito de descifrado homomórficamente.
# level_budget = [niveles_colapsando, niveles_elevando]
level_budget = [2, 2]
niveles_utiles = 2
profundidad = niveles_utiles + fhe.FHECKKSRNS.GetBootstrapDepth(
    level_budget, fhe.SecretKeyDist.UNIFORM_TERNARY
)
```

```
params.SetMultiplicativeDepth(profundidad)
params.SetScalingModSize(48)
params.SetFirstModSize(60)
num_slots = 4  # slots ocupados en el cifrotexto
params.SetBatchSize(num_slots)

cc = fhe.GenCryptoContext(params)
cc.Enable(fhe.PKESchemeFeature.PKE)
cc.Enable(fhe.PKESchemeFeature.KEYSWITCH)
cc.Enable(fhe.PKESchemeFeature.LEVELEDSHE)
cc.Enable(fhe.PKESchemeFeature.ADVANCEDSHE)
cc.Enable(fhe.PKESchemeFeature.FHE)

keys = cc.KeyGen()
cc.EvalMultKeyGen(keys.secretKey)
cc.EvalBootstrapSetup(level_budget, [0, 0], num_slots)
cc.EvalBootstrapKeyGen(keys.secretKey, num_slots)
```

La parametrización de OpenFHE es diferente a la de TenSEAL, pero los conceptos son los mismos. En TenSEAL especificábamos la cadena completa de módulos como una lista de tamaños en bits: `coeff_mod_bit_sizes=[60, 40, 40, 60]` — dos niveles intermedios de 40 bits cada uno. OpenFHE separa esa información: `SetMultiplicativeDepth` indica *cuántos* niveles hay (un entero), y `SetScalingModSize(48)` indica *cuántos bits* tiene cada nivel intermedio. `SetFirstModSize(60)` fija el tamaño del primer eslabón, igual que el 60 inicial en TenSEAL.

`SetBatchSize(num_slots)` indica cuántos *slots* ocupa el cifrotexto. Es necesario para que el contexto alinee correctamente las rotaciones internas del bootstrapping con la longitud del criptotexto.

La novedad aquí es el `level_budget` y `GetBootstrapDepth`. El bootstrapping consume niveles multiplicativos propios al evaluar internamente el circuito de descifrado: `level_budget = [2, 2]` indica cuántos niveles dedicar a cada una de sus dos fases internas — veremos qué hacen estas fases en Capítulo 5. `GetBootstrapDepth` toma estos presupuestos y calcula cuántos niveles extra necesita el circuito de bootstrapping en total, que se suman a los `niveles_utiles` del programador.

`EvalBootstrapSetup(level_budget, [0, 0], num_slots)` prepara el bootstrapping internamente. C`num_slots` debe coincidir con el `SetBatchSize` anterior. `EvalBootstrapKeyGen` genera las claves de rotación necesarias para el bootstrapping.

Por último, habilitamos `PKESchemeFeature.FHE` (FHE completo, no solo *leveled*) — sin esto, `EvalBootstrap` no estaría disponible.

Ahora ciframos los mismos datos de la Receta 4 y repetimos el experimento:

```
datos = [3.0, 7.0, 2.0, 5.0]
ptxt = cc.MakeCKKSPackedPlaintext(datos)
ctxt = cc.Encrypt(keys.publicKey, ptxt)

ctxt = cc.EvalMult(ctxt, ctxt)   # x² - consume 1 nivel
ctxt = cc.EvalMult(ctxt, ctxt)   # x  - consume otro nivel

# Niveles agotados. En la Receta 4 esto era el final.
# Aquí: bootstrapping al rescate.
ctxt = cc.EvalBootstrap(ctxt)

# ¡Ahora podemos seguir operando!
ctxt = cc.EvalMult(ctxt, ctxt)   # x

result = cc.Decrypt(ctxt, keys.secretKey)
result.SetLength(len(datos))
# → [6561.0, 5764801.0, 256.0, 390625.0]  (3 , 7 , 2 , 5 )
```

La llamada a `EvalBootstrap` es la que hace la magia: toma el cifrotexto agotado, evalúa homomórficamente el circuito de descifrado (el proceso que detallaremos en Capítulo 5) y produce un cifrotexto fresco con los niveles restaurados. La operación tarda del orden de un segundo — frente a los milisegundos que cuesta una multiplicación homomórfica corriente — y el cifrotexto resultante arrastra algo más de error de aproximación. Pero funciona: la multiplicación que en la Receta 4 lanzaba «scale out of bounds» ahora se completa sin problemas, e incluso podemos encadenar una tercera.

Preguntas de repaso

2.9. El bootstrapping restaura niveles multiplicativos pero tarda ~1 s frente a ~1 ms de una multiplicación normal. Si necesitas evaluar un circuito de profundidad 20, ¿qué estrategia es mejor: dimensionar 20 niveles desde el inicio, o usar 5 niveles con bootstrapping cada 5 multiplicaciones?

2.10. Pídele a un LLM: *«¿Qué es la seguridad circular en FHE y por qué es necesaria para el bootstrapping?»*

2.6 Receta 6: Búsqueda cifrada

Elena está satisfecha con los modelos de riesgo, pero ahora tiene otra necesidad: *«Necesito consultar el score de un paciente concreto en la base del proveedor. Pero no quiero que sepan qué pacientes me interesan — eso revelaría información clínica.»*

Esto es un problema de *recuperación privada de información* (*Private Information Retrieval* en inglés): obtener un registro de una base de datos sin que el servidor sepa cuál se pidió. Con FHE, la solución es sorprendentemente sencilla.

El truco consiste en cifrar un vector *one-hot* — un vector de ceros con un solo 1 en la posición del registro deseado. El servidor multiplica ese vector cifrado por su base de datos (en claro) y suma todos los productos. Dado que multiplicar por 0 cifrado «anula» todas las posiciones excepto la consultada, el resultado cifrado contiene únicamente el valor buscado. El servidor ha procesado *todos* los registros, pero solo uno ha sobrevivido a la multiplicación — y el servidor no sabe cuál.

```
# Base de datos del proveedor (en claro en su servidor)
scores = [85.2, 62.0, 91.7, 45.3, 78.9, 33.1, 70.4, 56.8]

# Elena quiere el score del paciente en posición 3,
# sin que el proveedor sepa que es el paciente 3
indice = 3
consulta = [1.0 if i == indice else 0.0
            for i in range(len(scores))]
consulta_cifrada = ts.ckks_vector(contexto, consulta)

# El servidor ejecuta esto - solo ve el cifrotexto
resultado_cifrado = (consulta_cifrada * scores).sum()

# Elena descifra
print(resultado_cifrado.decrypt()[0])  # → 45.3
```

La operación `consulta_cifrada * scores` es una multiplicación por constantes — no consume niveles multiplicativos, igual que en la Receta 3. Y `sum()` usa rotaciones, igual que en la Receta 1. No necesitamos ninguna operación nueva; solo combinamos las primitivas que ya conocemos de una forma diferente. Es un patrón que aparece una y otra vez en FHE: las mismas dos operaciones básicas (multiplicación y suma) dan lugar a aplicaciones muy distintas según cómo se diseñe el circuito.

Pero hay un coste ineludible: **el servidor debe procesar todos los registros para cada consulta**. No puede «saltarse» las posiciones que valen cero, porque no sabe cuáles son — el vector está cifrado. Si la base de datos tiene 10 000 registros, el servidor multiplica y suma los 10 000, aunque solo uno contenga información útil. Es el precio de la privacidad: la consulta se esconde precisamente porque es indistinguible de cualquier otra. Si el servidor pudiera procesar solo un subconjunto, podría deducir por eliminación qué posición se consultó.

Con $N = 8192$, cada cifrotexto almacena hasta $N_{\text{slot}} = 4096$ slots — suficiente para una base de datos de ese tamaño sin necesidad de múltiples cifrotextos. Para bases más grandes, el programador debe dividir los registros en bloques de N_{slot}, enviar un cifrotexto de consulta por bloque, y sumar los resultados parciales.

¿Y esto escala a bases de datos de millones de registros? No demasiado bien, y no es solo una cuestión de ingeniería: es una limitación de fondo del propio esquema. Repartir los registros en más bloques no reduce el trabajo total del servidor, solo lo redistribuye — el coste sigue creciendo de forma lineal con el tamaño de la base de datos, porque cada bloque adicional exige su propia multiplicación y suma sobre N_{slot} registros. Con 10 000 registros el retraso es imperceptible; con 10 millones, cada consulta puede tardar minutos incluso con las aceleraciones de hardware que veremos en Capítulo 4. Ninguna cantidad de *packing* SIMD cambia esta tendencia: solo reduce la constante, no el orden de crecimiento. Es el precio de que el servidor no pueda descartar ningún registro sin arriesgarse a filtrar cuál se consultó. En la práctica, los sistemas de recuperación privada a gran escala mitigan el problema particionando la base de datos por categorías públicas (por ejemplo, por hospital o por año), reduciendo así el universo de registros candidatos antes de aplicar esta técnica dentro de cada partición — a cambio de revelar esa categoría, que se asume no sensible.

Preguntas de repaso

2.11. En la búsqueda cifrada, ¿por qué el servidor debe procesar *todos* los registros para responder una sola consulta? ¿Qué información filtraría si procesara solo un subconjunto?

2.12. Si la base de datos del proveedor tiene 10 000 registros y $N_{\text{slot}} = 4096$, ¿cuántos cifrotextos necesita el cliente para una consulta? ¿Y cuántos debe procesar el servidor?

2.13. ¿Por qué dividir la base de datos en más bloques de N_{slot} no reduce el coste total del servidor? ¿Qué compromiso de privacidad asumen los sistemas de recuperación privada que particionan por categorías públicas?

Capítulo 3

Aplicaciones

El caso de Elena — analizar datos médicos en la nube sin exponerlos — no es un ejercicio teórico. Es exactamente el tipo de problema que ha motivado la inversión en FHE durante la última década: la externalización de infraestructura y software hacia terceros, incluso en sectores tan sensibles como la sanidad o la seguridad nacional, exige mantener la confidencialidad de los datos en todas las capas del sistema: tanto la infraestructura y el software como la información en tránsito. Veamos los escenarios donde el cifrado homomórfico aporta más valor.

Sanidad y datos clínicos. Un hospital quiere ejecutar modelos predictivos de diagnóstico sobre los historiales de sus pacientes usando la infraestructura de un proveedor en la nube. La regulación (HIPAA en EE. UU., RGPD en Europa) impide enviar datos médicos en claro a terceros. Las alternativas habituales son anonimizar los datos o añadir ruido estadístico mediante privacidad diferencial (el capítulo sobre privacidad diferencial). La anonimización es frágil: múltiples estudios han demostrado la re-identificación de registros médicos cruzando variables aparentemente inocuas. La privacidad diferencial, por su parte, degrada la precisión de los resultados al inyectar ruido, lo que puede ser inaceptable para diagnósticos individuales. Con FHE, el hospital cifra los datos antes de subirlos a la nube, el proveedor ejecuta sus modelos sobre los cifrotextos, y devuelve un resultado cifrado que solo el hospital puede descifrar. El proveedor nunca ve un solo diagnóstico.

Análisis financiero regulado. La detección de fraude bancario es más eficaz cuando múltiples instituciones comparten patrones de transacciones sospechosas. Pero compartir datos financieros entre bancos choca frontalmente con regulaciones de privacidad y con la reticencia competitiva. Hoy

se recurre a acuerdos de confidencialidad y entornos controlados, pero el banco receptor sigue viendo los datos en claro. El FHE permite que cada banco cifre sus transacciones y que un servicio centralizado ejecute algoritmos de detección sobre los datos cifrados de todos, sin que ninguna entidad acceda a los registros de las demás.

Machine Learning sobre datos sensibles. Entrenar un modelo de aprendizaje automático requiere acceso a grandes volúmenes de datos, que a menudo pertenecen a organizaciones distintas — hospitales, aseguradoras, administraciones públicas. FHE permite que varias partes contribuyan con datos cifrados a un entrenamiento conjunto sin revelar información individual. Esto abre la puerta a colaboraciones entre organizaciones que hoy son inviables por barreras legales o de confianza.

Búsquedas cifradas. Un usuario busca información en un servicio web, pero no quiere que el proveedor conozca sus consultas — piénsese en búsquedas de síntomas médicos o asesoramiento legal. Con FHE, la consulta se envía cifrada, el servidor la procesa contra su índice sin descifrarla, y devuelve resultados cifrados que solo el usuario puede leer. El proveedor ofrece el servicio sin saber qué se buscó.

Externalización segura de cómputo. Más allá de los sectores anteriores, cualquier organización que externalice cómputo a la nube enfrenta el mismo dilema: confiar en el proveedor o no usar la nube. El FHE elimina esa disyuntiva al permitir que los datos permanezcan cifrados durante todo el procesamiento. El proveedor nunca tiene acceso a los datos en claro, y además se puede combinar con pruebas de conocimiento cero (Lecturas Recomendadas) para que demuestre que ejecutó el cálculo correctamente — sin revelar nada sobre los datos ni sobre el proceso.

Preguntas de repaso

3.1. Para las aplicaciones de sanidad, finanzas y Machine Learning, ¿qué alternativa se usa hoy en la práctica y por qué es insatisfactoria?

3.2. Pídele a un LLM: *«¿Qué empresas o librerías ofrecen cifrado homomórfico en producción hoy? ¿Cuál es el estado de madurez?»* Verifica las respuestas con una búsqueda web.

Capítulo 4

Limitaciones actuales

Hemos visto que el FHE funciona y que tiene aplicaciones reales. Pero entre el concepto y el despliegue generalizado hay una serie de obstáculos que conviene entender con precisión. Las limitaciones están ordenadas de mayor a menor impacto práctico — desde las que más restringen qué productos se pueden construir y cómo se programan, hasta las que son mitigables con técnicas complementarias.

Rendimiento. Una operación individual sobre cifrotextos es entre 10^4 y 10^6 veces más lenta que su equivalente sin cifrar. Una multiplicación de dos enteros que un procesador moderno ejecuta en nanosegundos requiere milisegundos en FHE, porque implica multiplicar polinomios de grado N mediante la NTT — una transformada rápida que acelera la aritmética de polinomios (Capítulo 5) —, aplicar la *relinearización* — un paso que mantiene el cifrotexto en formato manejable tras cada multiplicación — y ejecutar el *rescaling* — el ajuste de escala que controla el crecimiento del error (Sección 5.4) —, todo ello sobre coeficientes de cientos de bits. Con $N = 8192$, cada operación elemental manipula decenas de kilobytes de datos algebraicos. Tres vías reducen esta brecha. Primero, el *packing* (Sección 5.6) amortiza el coste al procesar hasta $N/2$ valores en paralelo por operación — la ganancia de rendimiento más importante en la práctica. Segundo, instrucciones SIMD del procesador (AVX-512 vía Intel HEXL) y GPU aceleran la NTT y la aritmética modular, aportando entre $2\times$ y $100\times$ según la operación. Tercero, aceleradores hardware dedicados (FPGA y ASIC, como los proyectos DARPA DPRIVE) aspiran a ganancias de 10^3–$10^4\times$ adicionales, lo que cerraría la brecha a 10–$100\times$. Compiladores especializados (Zama Concrete, Google FHE transpiler) complementan

estas tres vías optimizando automáticamente el circuito aritmético. Aun con todas estas mejoras, el coste sigue siendo prohibitivo para aplicaciones de baja latencia — pero empieza a ser viable para cargas *batch* donde la privacidad justifica la sobrecarga.

Solo sumas y multiplicaciones. El esquema algebraico subyacente — llamado RLWE (*Ring Learning With Errors*, una variante eficiente de la criptografía de retículos basada en polinomios, que veremos en detalle en Capítulo 5) — solo proporciona dos operaciones nativas: suma y multiplicación de polinomios. Cualquier otra función — comparaciones, ordenación, ramificaciones condicionales, funciones no lineales como el coseno o la sigmoide — debe *aproximarse* mediante un polinomio de grado alto, y cada multiplicación de ese polinomio consume un nivel del presupuesto. Un simple `if (x > 0)` requiere evaluar un polinomio de grado 10 a 30, consumiendo entre 4 y 10 niveles multiplicativos. Esto hace que algoritmos que en claro son triviales — como encontrar el máximo de una lista — se conviertan en problemas de diseño de circuitos: hallar el máximo de n valores cifrados exige un torneo de comparaciones por parejas, con $\lceil \log_2 n \rceil$ rondas para reducir la lista a un único ganador, y cada ronda evalúa el polinomio de signo sobre todos los pares — entre 4 y 10 niveles por ronda. Con 1000 valores son 10 rondas y entre 40 y 100 niveles en total, muy por encima del presupuesto típico de 10 a 30 niveles sin recurrir al *bootstrapping* que veremos más abajo en este mismo apartado. El diseñador debe entonces elegir entre intercalar *bootstrappings* —a ~1 segundo cada uno— o cambiar a TFHE: un esquema basado en circuitos lógicos que representa los datos bit a bit y evalúa comparaciones como secuencias de puertas lógicas, sin aproximación polinómica, a costa de perder el paralelismo SIMD que hace rentable a CKKS para aritmética masiva (Capítulo 5).

Complejidad de programación. Programar sobre datos cifrados no se parece a programar en claro. El desarrollador debe diseñar el *circuito aritmético* — la secuencia completa de sumas y multiplicaciones que el esquema debe ejecutar — antes de empezar: planificar cuántas multiplicaciones encadenadas necesita, elegir los parámetros criptográficos en función de esa profundidad, empaquetar los datos para maximizar el paralelismo SIMD, y renunciar a cualquier ramificación dependiente de los datos (no se puede inspeccionar un valor cifrado para decidir qué rama tomar). Depurar es opaco: los cifrotextos intermedios no se pueden «imprimir» sin descifrarlos, lo que anula el propósito de la privacidad. Pero quizá el aspecto más traicionero — porque no produce errores visibles — es la *gestión de escalas*, un problema específico de CKKS. Los esquemas de aritmética exacta (BFV, BGV) también requieren gestionar niveles, pero un desajuste provoca un error explícito o es absorbido automáticamente por la librería — nunca corrompe el

resultado en silencio. En CKKS, en cambio, la codificación aproximada de números reales introduce un factor de escala Δ que se transforma con cada operación: una traducción matemáticamente correcta de una fórmula puede producir resultados absurdos si el cómputo agota el margen de precisión disponible en el último eslabón de la cadena de módulos — como vimos en la Receta 2 con la varianza cifrada, donde ambas ramas llegaban con escalas y parámetros internos idénticos y aun así desbordaban — o si distintas ramas terminan con escalas genuinamente incompatibles. En ninguno de los dos casos CKKS lanza error: simplemente interpreta mal los bits internos y devuelve basura. El programador debe rastrear manualmente cómo cada operación — multiplicación cifrado×cifrado, multiplicación por constante, rescaling — transforma la escala Δ del cifrotexto, y asegurarse de que todas las ramas llegan al mismo estado antes de combinarse. Compiladores como Concrete (Zama) y el FHE transpiler de Google automatizan parte de este proceso, transformando código de alto nivel en circuitos FHE optimizados, pero la tecnología de compilación está todavía en fase temprana.

Profundidad de circuito limitada. El presupuesto de niveles multiplicativos se fija *antes* de cifrar, mediante la cadena de módulos `coeff_mod_bit_sizes` (Sección 5.4). Si el circuito necesita más multiplicaciones de las presupuestadas, hay que redimensionar los parámetros (aumentando N y q, lo que encarece *todas* las operaciones) o recurrir al bootstrapping, que consume ~1 segundo y gasta parte de los niveles recién restaurados. No existe la posibilidad de «añadir niveles sobre la marcha» — la topología del circuito debe conocerse de antemano.

Resultados aproximados (CKKS). El esquema CKKS, el más usado para números reales, produce resultados con un error inherente del orden de 10^{-6} en configuraciones típicas. Para estadísticas y machine learning esto suele ser aceptable, pero no lo es para aplicaciones que requieren aritmética exacta (contabilidad, votación). Los esquemas BFV y BGV ofrecen aritmética exacta sobre enteros, pero no soportan números reales directamente. Además, CKKS tiene un matiz de seguridad particular: como los resultados descifrados llevan rastros del ruido interno del cifrotexto, un atacante que observe suficientes descifrados puede extraer información sobre la clave secreta. Las librerías actuales implementan mitigaciones — como añadir ruido adicional al descifrar —, pero el desarrollador debe tener presente una regla práctica: no exponer descifrados intermedios a terceros.

Expansión del cifrotexto. Como veremos al describir *RLWE* en Capítulo 5, cada cifrotexto es internamente un par de polinomios de N coeficientes módulo q, lo que implica un tamaño de $2 \times N \times \lceil \log_2 q \rceil$ bits. Con nuestros parámetros ($N = 8192$, módulo total $q \approx 2^{200}$ — suma de los 60+40+40+60

bits de `coeff_mod_bit_sizes`), eso da $2 \times 8192 \times 200/8 \approx 400$ KB. Un número de 8 bytes se convierte en un cifrotexto de ~400 KB — una expansión de 50 000×. El *packing* (Sección 5.6) reduce el coste amortizado a ~100 bytes por valor, pero las claves criptográficas son aún más voluminosas: las claves de Galois ocupan cientos de megabytes y las de bootstrapping pueden alcanzar varios gigabytes. Transmitir estas claves a un proveedor en la nube puede tardar minutos incluso con conexiones rápidas, y el consumo de memoria RAM del servidor se multiplica.

Tamaño de claves. La clave secreta es compacta (unos pocos kilobytes), pero el material criptográfico necesario para las operaciones homomórficas es enorme. La Tabla 5.2 en Sección 5.7 muestra las cuatro claves necesarias; las de relinearización y especialmente las de Galois — una por cada posición de rotación — ocupan cientos de megabytes con $N = 8192$. Si se añade bootstrapping, las claves de evaluación pueden superar el gigabyte. Esto limita los dispositivos donde se puede ejecutar FHE (un sensor IoT no puede almacenar claves de 500 MB) y añade latencia en escenarios donde las claves deben transmitirse al servidor.

Maleabilidad. Por definición, un esquema homomórfico permite que cualquiera transforme un cifrotexto en otro cifrotexto válido que cifra un mensaje relacionado — eso es precisamente lo que lo hace útil. Pero esa misma propiedad implica que los cifrotextos son *maleables*: un atacante podría modificar un resultado cifrado sin ser detectado (por ejemplo, sumar una constante al salario cifrado de alguien). Los esquemas FHE garantizan que un atacante no puede distinguir entre dos cifrotextos — no puede saber qué mensaje se cifró. Pero *no* garantizan que un atacante no pueda *modificar* un cifrotexto de forma útil sin ser detectado. En la práctica, la integridad del cómputo se protege combinando FHE con pruebas de conocimiento cero (Lecturas Recomendadas) o con esquemas de computación verificable.

Preguntas de repaso

4.1. Si las operaciones homomórficas son 10^4–10^6 veces más lentas, ¿para qué tipo de aplicaciones el coste es aceptable y para cuáles no?

4.2. Pídele a un LLM: *«¿Cómo se compara el rendimiento del FHE en 2024 con el de 2015? ¿Cuáles fueron los avances principales?»*

4.3. Un ingeniero necesita implementar una comparación `x > y` sobre datos cifrados con FHE. ¿Por qué no puede hacerlo directamente? ¿Qué alternativa tiene y cuál es su coste?

Capítulo 5

Fundamentos

Daniel ya ha podido experimentar con la tecnología —lo que él llama, quitándole hierro, «jugar con ella»—, y para Elena los resultados bastan para seguir adelante con el proyecto: el proveedor tendrá su análisis y legal su confidencialidad. Pero ¿es este nivel de entendimiento suficiente para mantener una tecnología nueva en un sistema en producción? Daniel y su equipo tendrán que ocuparse de mantenerla, extenderla cuando el negocio lo pida, y explicar —cuando algo falle y no pueda arreglarse de inmediato— las razones fundamentales por las que falla. Finalmente, para no quedarse obsoletos, tendrán que estar atentos a qué innovaciones pueden tener un impacto real en rendimiento, funcionalidad o coste. ¿Qué necesita saber para todo eso?

Hemos visto que el código funciona y que las aplicaciones potenciales son reales, pero a costa de un rendimiento que por operación elemental en CPU es entre 10^4 y 10^6 veces peor que en claro. Aunque técnicas como el *packing* (Sección 5.6), las GPU y los futuros aceleradores hardware pueden recortar notablemente esa brecha, la sobrecarga sigue siendo muy elevada. ¿Cómo es posible que un ordenador ejecute cálculos sobre datos que no puede ver, y por qué es tan caro incluso con estas optimizaciones? Recordemos la analogía de la caja con guantes para manipular joyas o la de cocinar sin tocar los ingredientes: Alice mete sus materiales dentro de una caja cerrada con llave, y un trabajador los manipula a través de los guantes sin tocarlos jamás. En FHE, los datos son los materiales, el cifrado es la caja, y las operaciones homomórficas son los guantes. Vamos a ver cómo se construye esa «caja» matemáticamente, y de paso entenderemos cada parámetro del código que acabamos de ejecutar. La clave está en una rama de las matemáticas que poca gente asocia con criptografía: los retículos.

5.1 Retículos

La seguridad de los esquemas de cifrado homomórfico no depende de la factorización de números primos — como RSA — sino de la dificultad de resolver problemas geométricos en estructuras llamadas *retículos*. La comprensión de estas estructuras es lo que le permitirá a Daniel responder con certeza al departamento legal si los datos de los pacientes están a salvo en la nube.

¿Y qué es un retículo? Un retículo (*lattice*) es un conjunto discreto de puntos en un espacio N-dimensional, generado por combinaciones lineales enteras de una base de vectores $B = \{b_1, \dots, b_N\}$:

$$L(B) = \left\{ \sum_{i=1}^{N} a_i b_i : a_i \in \mathbb{Z} \right\}$$

En general, los vectores base b_i pueden ser reales, pero en criptografía trabajamos siempre con vectores de enteros módulo q — es decir, $b_i \in \mathbb{Z}_q^N$ — de modo que todos los puntos del retículo también son enteros. Piensa en una cuadrícula regular como en Figura 5.1, pero en cientos de dimensiones (véase el capítulo sobre criptografía de retículos para un tratamiento completo).

La criptografía moderna se apoya en dos problemas fundamentales sobre estas estructuras sobre bases de vectores enteros:

- **SVP** (*Shortest Vector Problem*): hallar el vector no nulo más corto del retículo.
- **CVP** (*Closest Vector Problem*): dado un punto cualquiera del espacio, hallar el punto del retículo más cercano. Si se exige el punto *exacto* más cercano, el problema es NP-hard. En su versión aproximada ya no es NP-hard, pero los mejores algoritmos conocidos siguen teniendo un coste exponencial en la dimensión N del retículo.

CVP es al menos tan difícil como SVP, y ambos son la base de la criptografía de retículos. En criptografía se usan precisamente las versiones aproximadas, lo bastante relajadas para no ser NP-hard, pero lo bastante estrictas para que ningún algoritmo conocido, clásico o cuántico, las resuelva en tiempo razonable. En las dimensiones típicas de FHE ($N \geq 1024$), el problema es intratable, y esta dificultad es la que protege los datos. Pero un retículo por sí solo no cifra nada — necesitamos traducir esta dificultad geométrica en un mecanismo que permita cifrar y operar.

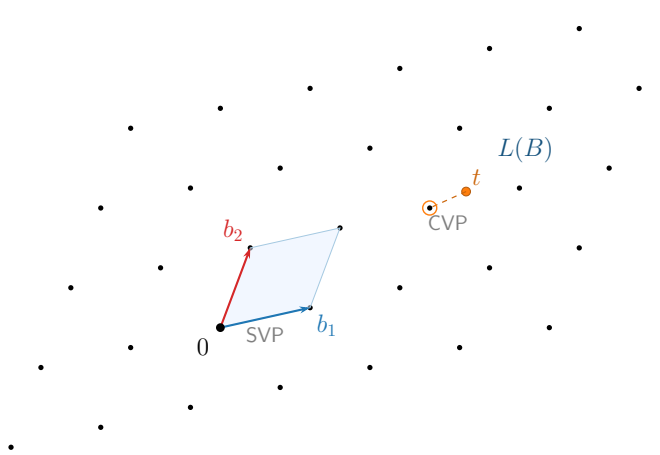


Figura 5.1: Retículo bidimensional generado por dos vectores base b_1 y b_2. La región sombreada es el *dominio fundamental* (paralelogramo definido por la base). El punto t está fuera del retículo: encontrar el punto del retículo más cercano (CVP) o el vector no nulo más corto (SVP) es computacionalmente intratable en dimensiones altas.

Preguntas de repaso

5.1. ¿Cuál es la diferencia entre SVP y CVP? ¿Por qué CVP es al menos tan difícil como SVP?

5.2. La seguridad de FHE se basa en que los problemas de retículos son difíciles «incluso para ordenadores cuánticos». ¿Qué distingue estos problemas de los que usa RSA (factorización) frente a ataques cuánticos?

5.2 De los retículos a los anillos de polinomios: LWE y RLWE

Una vez comprendido que la seguridad reside en la dificultad de encontrar puntos en un retículo, necesitamos una forma de implementarlo eficientemente — la misma eficiencia que permite que el código de Daniel calcule la media salarial de Elena en milisegundos y no en horas. En esta sección veremos cómo el esquema *Learning With Errors* (LWE) traslada la geometría de retículos al álgebra lineal, y cómo dar el salto al *Ring Learning*

With Errors (RLWE), basado en polinomios, para mejorar la eficiencia del proceso de cifrado y descifrado sin comprometer la seguridad.

En este caso, la clave secreta s es un vector de N enteros y la parte pública incluye una matriz aleatoria A de $N \times N$ enteros. La relación fundamental entre la clave privada y la clave pública consiste en calcular:

$$b = A \cdot s + e \pmod{q}$$

donde e es un vector de errores pequeños — ruido inyectado deliberadamente. A partir de esta estructura, cifrar un mensaje m (codificado como vector) consiste en añadirlo directamente sobre el valor enmascarado:

$$c = A \cdot s + e + m \pmod{q}$$

Sin ese ruido e, recuperar s a partir de A y b sería álgebra lineal de primer curso (eliminación gaussiana), y obtener m a partir de c resultaría inmediato. Pero con el error, b es un punto *cercano* al retículo generado por las columnas de A, pero no exactamente *en* el retículo — y encontrar el punto más cercano es CVP. Esa es la trampa: el ruido convierte un problema trivial en uno intratable.

LWE funciona, pero tiene un problema de rendimiento: cada operación sobre un cifrotexto implica un producto matriz-vector — $O(N^2)$ multiplicaciones de enteros. Con las dimensiones necesarias para FHE ($N \geq 1024$), eso son millones de operaciones por cada suma o multiplicación homomórfica. Además, la matriz A tiene N^2 enteros: con $N = 8192$, almacenarla requiere más de 67 millones de enteros. Demasiado lento y pesado para ser práctico.

La solución consiste en reemplazar vectores y matrices por **polinomios**. Un polinomio de grado menor que N — por ejemplo, $3x^2 + 5x + 1$ — no es más que un vector de N coeficientes $(1, 5, 3, 0, \ldots, 0)$ escrito con otra notación. Hasta aquí no hemos ganado nada. Pero si restringimos los polinomios a un *anillo* concreto ocurre algo notable.

¿Pero qué es un anillo y qué significa aplicar una restricción a un polinomio? Un anillo es, simplemente, un conjunto de elementos —aquí, polinomios— junto con una suma y una multiplicación que se pueden combinar libremente sin salirse nunca del propio conjunto: sumar o multiplicar dos elementos del anillo siempre produce otro elemento del anillo. $\mathbb{Z}_q$, los enteros módulo q que ya usamos en LWE, es el ejemplo más simple: sumar o multiplicar dos enteros módulo q da otro entero módulo q, nunca se «escapa» del rango

$[0, q-1]$. Nuestro anillo de polinomios necesita esa misma propiedad de cierre, y para conseguirla en notación algebraica se escribe $R_q = \mathbb{Z}_q[x]/(x^N+1)$: polinomios con coeficientes enteros módulo q (i.e., $\mathbb{Z}_q[x]$, la misma aritmética modular de LWE), con la restricción adicional de que $x^N + 1 = 0$, es decir, $x^N = -1$. Esta restricción sirve para que el grado nunca crezca: cuando al multiplicar dos polinomios algún término supera el grado $N-1$ —y por tanto se saldría del anillo—, la regla lo «pliega» de vuelta al rango $[0, N-1]$ como cuando aplicamos la operación módulo a un entero. Veamos un ejemplo con $N = 4$ y $q = 7$. Partimos de dos polinomios en R_7:

$$a(x) = 2x^3 + x + 3, \qquad b(x) = x^3 + 4x^2 + 1$$

Al multiplicarlos aparecen términos de grado superior a 3. El truco del anillo es aplicar $x^4 = -1$: por ejemplo, $2x^3 \cdot x^3 = 2x^6 = 2 \cdot x^2 \cdot x^4 = -2x^2 = 5x^2 \pmod 7$. Tras plegar todos los términos, el resultado es $2x^3 + 3x^2 + 2$ —siempre un polinomio de 4 coeficientes, igual que los vectores de LWE. Lo crucial es que esta restricción tiene exactamente el mismo efecto que restringir el conjunto de posibles matrices A al de matrices *anticirculantes* (cuyas filas son rotaciones de un único vector con signos alternados). Es decir, toda la información de la matriz A queda codificada en los coeficientes de un solo polinomio $a(x)$.

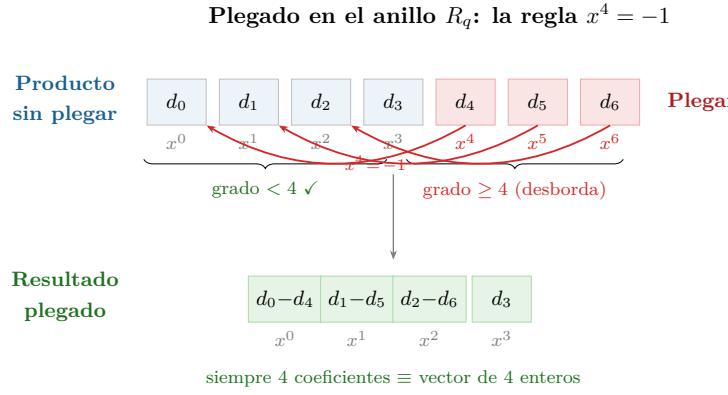


Figura 5.2: Plegado en el anillo R_q. Al multiplicar polinomios, los términos de grado mayor o igual que N se pliegan al rango $[0, N-1]$ mediante la regla $x^N = -1$. Esta restricción equivale a trabajar con matrices anticirculantes, codificando una matriz de $N \times N$ enteros en solo N coeficientes.

El resultado es *Ring Learning With Errors* (RLWE): la versión de LWE donde vectores y matrices se reemplazan por polinomios en R_q. Como antes en el esquema algebraico de LWE, una sola ecuación pero esta vez polinómica codifica la relación entre la clave pública y la clave privada:

$$b(x) = a(x) \cdot s(x) + e(x) \pmod{q,\ x^N + 1}$$

La ganancia es doble. Primero, en almacenamiento: la matriz A de LWE tiene N^2 enteros; el polinomio $a(x)$ de RLWE tiene solo N. Con $N = 8192$, pasamos de 67 millones de enteros a 8192 — una reducción de 8 000× en el tamaño de las claves. Segundo, en velocidad: multiplicar una matriz por un vector cuesta $O(N^2)$; multiplicar dos polinomios en R_q cuesta solo $O(N \log N)$ gracias a la *Number Theoretic Transform* (NTT), la versión modular de la FFT (*Fast Fourier Transform*). Con $N = 8192$, eso es ~100,000 operaciones frente a ~67 millones — tres órdenes de magnitud más rápido. Esta eficiencia es lo que hace el FHE viable.

El nombre RLWE resume el mecanismo. *Ring* se refiere al anillo de polinomios — la fuente de la eficiencia. *Learning With Errors* describe el problema subyacente: dado $b = a \cdot s + e$, ¿se puede «aprender» el secreto s? Sin el error, bastaría con álgebra lineal. Con él, recuperar s equivale a resolver CVP en un retículo formado por polinomios — lo que enseguida llamaremos un *retículo ideal* — justo la dificultad de la subsección anterior.

Conviene precisar que este salto de LWE a RLWE no sale gratis en términos de seguridad. LWE tiene una garantía teórica al estar basada sobre retículos arbitrarios; RLWE, en cambio, solo tiene esa garantía al estar basada sobre retículos restringidos a la estructura del anillo de polinomios R_q (lo que se conoce como *retículos ideales*). Este grado adicional de estructura es precisamente lo que habilita el uso de la NTT y su ganancia de rendimiento, pero también podría, en principio, darle a un atacante un punto de apoyo que no existe en el caso general. En la práctica no se conoce ningún ataque que explote esta estructura para las familias de anillos usadas (potencias de dos, como $x^{8192} + 1$), así que RLWE se considera tan seguro como LWE — pero formalmente descansa en una hipótesis distinta y más específica.

CKKS, el esquema utilizado en la sección práctica (Capítulo 2), se construye directamente sobre RLWE, y la conexión es profunda. A diferencia de otros esquemas que tratan el error e como algo que debe eliminarse por completo, CKKS lo interpreta como un *error de precisión* — similar a lo que ocurre en la aritmética de punto flotante. Como RLWE ya incluye un error inherente, el esquema lo reutiliza para representar números reales como valores aproximados: el ruido criptográfico y el error de aproximación

se gestionan juntos mediante las técnicas de re-escalado que veremos en Sección 5.4.

Con la estructura matemática lista — anillo eficiente, ruido que garantiza la seguridad, NTT que acelera las operaciones —, falta la pieza central: ¿cómo se cifra un mensaje concreto, y por qué sumar o multiplicar cifrotextos produce el resultado correcto?

El compromiso de N: seguridad, paralelismo y memoria

El parámetro `poly_modulus_degree` que vimos en el código de Capítulo 2 fija exactamente la dimensión N del grado del polinomio del anillo R_q. Como dijimos, debe ser potencia de 2 (4096, 8192, 16384…) para que la NTT — como la FFT — funcione correctamente. La elección de N implica un compromiso: un N mayor da más seguridad, al tener una clave de mayor longitud, y también más *slots* de paralelismo ($N_{\text{slot}} = N/2$ valores por cifrotexto, como veremos en Sección 5.6). El problema es que, a medida que aumenta la dimensión del polinomio, sus operaciones se vuelven más caras, escalando como $O(N \log N)$: duplicar N de 8192 a 16384 duplica N_{slot}, pero también duplica (aproximadamente) el tiempo de cada suma, multiplicación o rotación.

Esta representación polinómica tiene además un coste directo en memoria. Cada cifrotexto es un par de polinomios (a, b), cada uno con N coeficientes enteros módulo q. El tamaño total es $2 \times N \times \lceil \log_2 q \rceil$ bits. Con los parámetros de nuestro ejemplo ($N = 8192$ y un módulo total $q \approx 2^{200}$, suma de los 60+40+40+60 bits de `coeff_mod_bit_sizes`), un solo cifrotexto ocupa aproximadamente $2 \times 8192 \times 200/8 \approx 400$ KB. Un *double* en claro ocupa 8 bytes; su versión cifrada, 400,000 bytes — una expansión de $50\,000\times$. El *packing* (Sección 5.6) amortiza este coste al empaquetar hasta $N_{\text{slot}} = N/2 = 4096$ valores en un solo cifrotexto, reduciendo el coste a unos 100 bytes por valor. Pero las claves criptográficas adicionales — especialmente las de Galois y las de bootstrapping — pueden alcanzar cientos de megabytes o incluso gigabytes, como veremos más adelante.

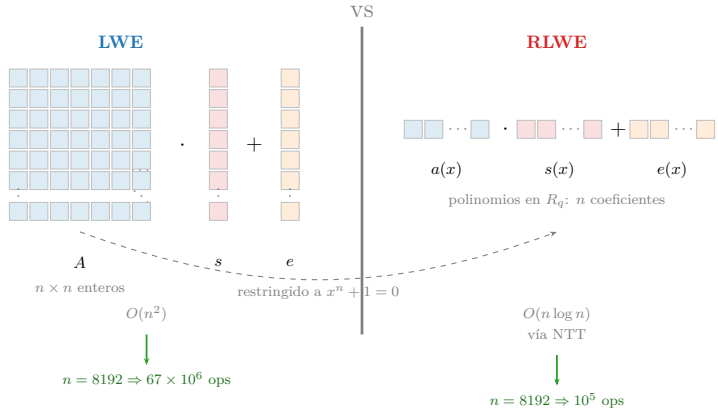


Figura 5.3: LWE frente a RLWE. A la izquierda, LWE requiere una matriz A de $N \times N$ enteros. A la derecha, RLWE codifica la misma información en un polinomio de N coeficientes, reduciendo el coste de $O(N^2)$ a $O(N \log N)$ vía la NTT.

Preguntas de repaso

5.3. En LWE estándar, la parte pública es una matriz A de $N \times N$ enteros. En RLWE se reemplaza por un solo polinomio de N coeficientes. ¿Qué propiedad algebraica del anillo R_q permite esta compresión? ¿Qué ganancia en velocidad proporciona?

5.4. Pídele a un LLM: *«En criptografía basada en retículos, ¿por qué se considera que resolver RLWE es al menos tan difícil como resolver el problema del vector más cercano (CVP)? Explica la reducción de forma intuitiva.»* Verifica que mencione el papel del ruido e en la conexión con CVP.

5.5. Trabaja en el anillo $R_5 = \mathbb{Z}_5[x]/(x^4 + 1)$. Multiplica $a(x) = 3x^3 + 2x^2 + 1$ por $b(x) = x^2 + 2x$, pliega los términos de grado ≥ 4 con la regla $x^4 = -1$ y reduce los coeficientes módulo 5. ¿Cuál es el polinomio resultante?

5.3 Cifrado y operaciones homomórficas

Ya tenemos la estructura algebraica — el anillo de polinomios R_q con sus operaciones eficientes vía NTT — y sabemos que la seguridad reside en la

dificultad de resolver problemas de retículos. Ahora veamos cómo se usa esta maquinaria para cifrar mensajes y, sobre todo, por qué las operaciones aritméticas sobre cifrotextos producen resultados correctos sin necesidad de descifrar — exactamente lo que Daniel comprobó de forma empírica en la Receta 1, cuando la media salarial cifrada coincidió con la calculada en claro.

La mecánica del cifrado es elegante. La clave secreta s es un polinomio con coeficientes pequeños. Para cifrar un mensaje m, se genera un polinomio aleatorio a — a diferencia de s y e, con coeficientes uniformemente distribuidos en todo el rango módulo q, no pequeños — y un *error* pequeño e — ruido deliberadamente inyectado para hacer el problema de descifrado más difícil — y se produce un par (a, b):

$$\text{Enc}(m) = (a,\ b = a \cdot s + e + m)$$

¿Por qué hace falta a? ¿No bastaría con cifrar como $c = s+e+m$ y descifrar como $c - s$? El problema es que sin a, si alguien intercepta dos cifrotextos $c_1 = s + e_1 + m_1$ y $c_2 = s + e_2 + m_2$, puede restarlos: $c_1 - c_2 \approx m_1 - m_2$. La clave s se cancela y el atacante aprende la diferencia entre los mensajes. El polinomio aleatorio a, distinto en cada cifrado, actúa como una máscara que hace cada cifrotexto indistinguible de ruido aleatorio. Quien conoce s puede descifrar el cifrotexto c — $\text{Dec}(c) = b - a \cdot s = e + m$ — y recupera el mensaje en claro m (el error e es pequeño y se puede obviar como un error de precisión). Sin conocer s, la presencia de a y e hace que recuperar m sea equivalente a resolver un problema de retículos.

La propiedad homomórfica surge de forma natural. Veamos por qué. Sean dos cifrotextos $c_1 = (a_1, b_1)$ y $c_2 = (a_2, b_2)$ que cifran m_1 y m_2 respectivamente, donde $b_i = a_i \cdot s + e_i + m_i$.

Suma. Basta sumar componente a componente:

$$\text{Enc}(m_1 + m_2) = c_1 + c_2 = (a_1 + a_2,\ b_1 + b_2)$$

Si descifras este resultado con la clave s, obtienes:

$$\begin{aligned} \text{Dec}(c_1 + c_2) &= (b_1 + b_2) - (a_1 + a_2) \cdot s \\ &= (e_1 + e_2) + (m_1 + m_2) \end{aligned}$$

Es decir, el mensaje descifrado es m_1+m_2, la suma esperada de los mensajes en claro, con un error de (e_1+e_2). El error crece, pero solo como una suma — si cada e_i es pequeño, la suma sigue siendo pequeña.

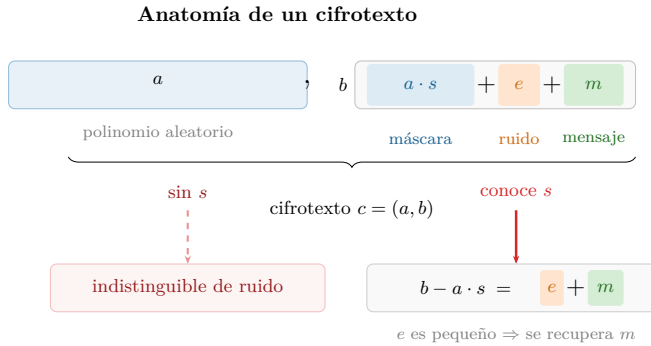


Figura 5.4: Anatomía de un cifrotexto. El par (a, b) contiene el mensaje m oculto tras una máscara $a \cdot s$ y un ruido e. Quien conoce s elimina la máscara y recupera m; sin s, el cifrotexto es indistinguible de ruido aleatorio.

Suma con constante en claro. ¿Qué ocurre si uno de los operandos no está cifrado? Sea $c = (a, b)$ un cifrotexto que cifra m, y k una constante en claro. La suma es aún más sencilla:

$$\text{Enc}(m + k) = c + k = (a,\ b + k)$$

Solo se modifica la segunda componente. Al descifrar:

$$\text{Dec}(c + k) = (b + k) - a \cdot s = e + m + k$$

El error e no cambia en absoluto — la constante se suma directamente al mensaje sin afectar al ruido. Esta es la operación más barata del repertorio: ni siquiera hay crecimiento de error.

Multiplicación por constante. Ahora multipliquemos el cifrotexto $c = (a, b)$ por una constante en claro k:

$$\text{Enc}(k \cdot m) = k \cdot c = (k \cdot a,\ k \cdot b)$$

Al descifrar:

$$\begin{aligned} \text{Dec}(k \cdot c) &= k \cdot b - k \cdot a \cdot s = k \cdot (e + m) \\ &= k \cdot m + k \cdot e \end{aligned}$$

El mensaje es $k \cdot m$ — correcto — y el error pasa de e a $k \cdot e$, proporcional a la constante. El ruido crece, pero de forma controlada. Además, multiplicar por una constante simplemente escala ambas componentes del par: la estructura (a, b) se preserva intacta, sin complicaciones adicionales. Veremos enseguida que la multiplicación entre *dos cifrotextos* es mucho más costosa — requiere maquinaria extra y consume parte del presupuesto del esquema.

Esto explica por qué en las recetas anteriores se insistía en que operaciones como `features_cifrados * pesos` (Receta 3) o `consulta_cifrada * scores` (Receta 6) eran «baratas»: al multiplicar un cifrotexto por valores en claro, el coste es mínimo. Solo la multiplicación entre *dos cifrotextos* — que veremos a continuación — tiene un coste significativo.

Multiplicación entre cifrotextos. Las operaciones con constantes en claro eran directas — sumas y multiplicaciones componente a componente, sin coste adicional. Para la multiplicación entre dos cifrotextos no es tan sencilla.

Queremos definir $\mathrm{Enc}(m_1 \cdot m_2) = c_1 \otimes c_2 = (a, b)$ — usamos $\otimes$ en lugar de $\cdot$ porque, como veremos, esta operación sobre cifrotextos es estructuralmente distinta de la multiplicación por constantes — tal que $\mathrm{Dec}(c_1 \otimes c_2) = b - a \cdot s \approx m_1 \cdot m_2$. La pregunta es: ¿qué deben ser a y b?

Para descubrirlo, trabajamos hacia atrás. Sabemos que descifrar cada cifrotexto da $b_i - a_i \cdot s = m_i + e_i$, así que el producto de ambos descifrados es $(m_1 + e_1)(m_2 + e_2) \approx m_1 \cdot m_2$ — justo lo que queremos. Expandimos $(b_1 - a_1 s)(b_2 - a_2 s)$ para ver qué forma tiene:

$$(b_1 - a_1 s)(b_2 - a_2 s) = \underbrace{b_1 b_2}_{d_0} - \underbrace{(a_1 b_2 + a_2 b_1)}_{d_1} \cdot s + \underbrace{a_1 a_2}_{d_2} \cdot s^2$$

Los tres coeficientes — $d_0 = b_1 b_2$, $d_1 = a_1 b_2 + a_2 b_1$ y $d_2 = a_1 a_2$ — se calculan exclusivamente con las componentes públicas, sin necesidad de s. Hasta aquí todo bien.

Pero hay un problema: la expresión no tiene la forma $b - a \cdot s$ que necesitamos para descifrar con un par. Tiene *tres* términos y depende de s^2. No podemos encontrar un par (a, b) que encaje directamente — necesitamos eliminar ese s^2.

Aquí entra la *relinearización*. Se publica de antemano una *clave de relinearización*: un cifrado de s^2 bajo la propia clave s, es decir, un par (a_r, b_r) donde $b_r \approx a_r \cdot s + s^2$. Con ella, absorbemos el término $d_2 \cdot s^2$ y construimos el par que buscábamos:

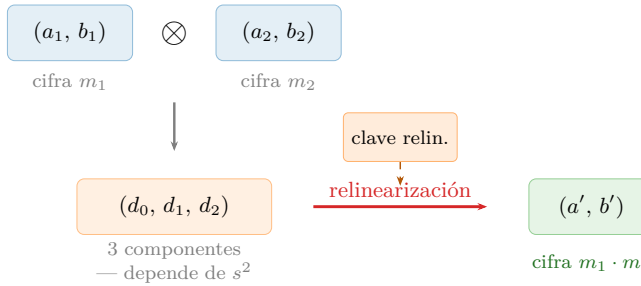


Figura 5.5: Multiplicación entre cifrotextos. El producto genera un triplete (d_0, d_1, d_2) que depende de s^2. La relinearización, usando una clave pública que cifra s^2, lo reduce a un par estándar (a', b') descifrable con s.

$$a = d_1 + d_2 \cdot a_r, \qquad b = d_0 + d_2 \cdot b_r$$

¿Funciona? Comprobemos: $b - a \cdot s = d_0 + d_2 b_r - (d_1 + d_2 a_r)s = d_0 - d_1 s + d_2(b_r - a_r s) \approx d_0 - d_1 s + d_2 s^2$, que es exactamente el producto original. Ahora sí tenemos un par con la forma estándar que se descifra solo con s y produce $m_1 \cdot m_2$.

Esto resuelve la pregunta que nos planteábamos: la multiplicación homomórfica es $\text{Enc}(m_1 \cdot m_2) = c_1 \otimes c_2 = (a, b)$ con a y b definidos arriba. En librerías como TenSEAL la relinearización se aplica internamente cada vez que multiplicas dos cifrotextos, así que el usuario solo escribe `c1 * c2`.

Pero una vez que somos capaces de multiplicar, aparece un segundo problema: el error crece mucho más rápidamente. Fijémonos que si sustituimos $b_i = a_i s + e_i + m_i$, el producto descifrado es:

$$(e_1 + m_1)(e_2 + m_2) = m_1 \cdot m_2 + e_1 \cdot m_2 + e_2 \cdot m_1 + e_1 \cdot e_2$$

El resultado contiene $m_1 \cdot m_2$ — lo que buscábamos — pero el error resultante $e_{\text{mult}} = e_1 \cdot m_2 + e_2 \cdot m_1 + e_1 \cdot e_2$ es mucho mayor que en la suma. Los errores ya no se *suman* sino que se *multiplican* con los mensajes, creciendo mucho más rápido. Fíjate que los dos primeros términos, $e_1 \cdot m_2$ y $e_2 \cdot m_1$, son exactamente el mismo crecimiento que vimos en la multiplicación por constante — el error de un cifrotexto escalado por el mensaje del otro —, solo que aquí ocurre en las dos direcciones a la vez, y el tercer término, $e_1 \cdot e_2$, es insignificante frente a los otros dos porque ambos errores son pequeños.

Así que, cuando mensajes y errores son de magnitud comparable, la multiplicación entre cifrotextos no es un mecanismo de crecimiento distinto: es, aproximadamente, el doble de una sola multiplicación por constante, por sumarse las dos contribuciones simétricas. Aun así, ese factor 2 —constante y modesto— no es lo que preocupa: el problema real es que el error crece multiplicándose con los mensajes en cada nivel del circuito, así que tras pocas multiplicaciones *encadenadas* el error acumulado puede superar al mensaje y el descifrado produce basura. Esta es la razón fundamental por la que el FHE tiene un presupuesto limitado de operaciones.

Qué es este presupuesto de ruido y cuántas operaciones se pueden hacer antes de sobrepasarlo será el foco de la siguiente sección.

i Solo suma y multiplicación

Conviene hacer explícito un punto sutil: las dos únicas operaciones que el esquema proporciona de forma nativa son la suma y la multiplicación de polinomios. La estructura algebraica de RLWE — un anillo de polinomios — solo define estas dos operaciones. Cualquier otro cálculo — comparaciones ($x > y$), máximos, mínimos, funciones no lineales como sigmoide o ReLU — debe *aproximarse* mediante un polinomio de grado suficientemente alto, y cada multiplicación de ese polinomio consume un nivel del presupuesto. Un simple `if (x > 0)` que en código normal es una instrucción, en FHE requiere evaluar un polinomio de grado 10 a 30 que aproxime la función signo, consumiendo entre 4 y 10 niveles multiplicativos. Esta restricción — a veces llamada modelo de *circuito aritmético* — es la que convierte la programación sobre datos cifrados en un ejercicio de diseño cuidadoso, como vimos en Capítulo 4.

Preguntas de repaso

5.6. En el cifrado RLWE, ¿por qué es necesario el polinomio aleatorio a en cada cifrado? ¿Qué ataque permite su ausencia?

5.7. Explica con tus palabras por qué la suma homomórfica es «barata» (el error crece linealmente) mientras que la multiplicación es «cara» (el error crece mucho más rápido).

5.8. ¿Qué papel juega la *relinearización* después de una multiplicación homomórfica? ¿Qué pasaría si no se aplicara?

5.9. CKKS no soporta ramas condicionales. ¿Cómo implementarías la siguiente lógica utilizando CKKS? ¿Cuántos niveles multiplicativos consume aproximadamente?

```
if (m < 0) then m = -m
```

5.4 Ruido y niveles

Acabamos de ver que cada multiplicación entre cifrotextos o entre cifrotextos y escalares hace crecer el error de forma dramática — los errores se multiplican con los mensajes en lugar de simplemente sumarse. Y adelantamos que, tras pocas multiplicaciones encadenadas, el error puede superar al mensaje y el descifrado produce basura. Pero ¿cuántas operaciones podemos encadenar exactamente, y cómo controla el esquema ese deterioro? Es justo lo que a Daniel le habría gustado saber antes de que Elena le llamara un viernes a las cinco de la tarde para decirle que «los números se desbordaban» (Receta 4).

La clave está en el módulo q. Todas las operaciones aritméticas del esquema se realizan módulo un número grande q, llamado *módulo del cifrotexto*. Tras cifrar un mensaje fresco, el error e es pequeño — del orden de unos pocos bits. El descifrado funciona correctamente siempre que el error acumulado sea menor que $q/2$. Podemos pensar en $\log_2(q)$ — el número de bits de q — como el tamaño del «depósito de combustible», y en $\log_2(e)$ como el combustible ya consumido.

¿Cuánto crece el error en cada multiplicación? En CKKS, los mensajes no se almacenan directamente sino escalados por un factor Δ (en nuestro código, $\Delta = 2^{40}$) — ya desde el momento del cifrado, no solo como efecto de multiplicar. Un cifrotexto recién cifrado que almacena el mensaje m contiene internamente $\Delta \cdot m + e$, donde e es el error. Al multiplicar dos cifrotextos:

$$(\Delta\, m_1 + e_1)(\Delta\, m_2 + e_2) = \Delta^2\, m_1 m_2 + \Delta(m_1 e_2 + m_2 e_1) + e_1 e_2$$

El término deseado es $\Delta^2 m_1 m_2$ y el error es $\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2$. Dado que e_1, e_2 son pequeños, el término $e_1 e_2$ es despreciable y el error dominante es $\Delta(m_1 e_2 + m_2 e_1)$. Hay dos problemas: la escala del mensaje ha pasado de Δ a Δ^2, y el error se ha multiplicado por Δ.

El *rescaling* resuelve ambos de un golpe: dividimos todo el cifrotexto por Δ:

$$\frac{\Delta^2 m_1 m_2 + \Delta(m_1 e_2 + m_2 e_1)}{\Delta} = \Delta\, m_1 m_2 + (m_1 e_2 + m_2 e_1)$$

Tras el rescaling, el mensaje vuelve a estar escalado por Δ — la forma estándar — y el error es ahora $e_{\text{resc}} = m_1 e_2 + m_2 e_1$. Los 40 bits extra de error que había introducido la multiplicación se han eliminado al dividir por $\Delta = 2^{40}$. El error resultante es mayor que los originales e_1, e_2 (por el factor m_i), pero ya no crece de forma catastrófica. Sin rescaling, cada multiplicación añadiría 40 bits de error y en dos o tres operaciones se desbordaría q; con rescaling, el error crece moderadamente y lo que se consume es un eslabón del módulo.

Ejemplo numérico: rescaling paso a paso

Veámoslo de forma simplificada con números concretos en vez de polinomios — con un $\Delta = 1000$ en vez de 2^{40} la cuenta se puede seguir a mano. Sean $m_1 = 2.5$, $m_2 = 3.0$, con errores $e_1 = 2$ y $e_2 = -3$. Cada cifrotexto fresco almacena internamente $\Delta m + e$:

$$\Delta m_1 + e_1 = 1000 \times 2.5 + 2 = 2502, \qquad \Delta m_2 + e_2 = 1000 \times 3.0 - 3 = 2997$$

Multiplicar los cifrotextos equivale a multiplicar estos dos enteros: $2502 \times 2997 = 7\,498\,494$. Comparado con el valor «ideal» $\Delta^2 m_1 m_2 = 1000^2 \times 2.5 \times 3.0 = 7\,500\,000$, la diferencia es -1506 — exactamente $\Delta(m_1 e_2 + m_2 e_1) + e_1 e_2 = 1000 \times (-1.5) + (-6) = -1506$, la fórmula de arriba comprobada con números reales.
Ahora el rescaling: dividimos por $\Delta = 1000$ y redondeamos al entero más cercano:

$$7\,498\,494 \;/\; 1000 = 7498.494 \;\approx\; 7498$$

Ese 7498 es el cifrotexto rescalado, ya de vuelta a la escala estándar $\Delta = 1000$ — la misma magnitud que un cifrotexto recién cifrado, listo para otra multiplicación. Al descifrar, $7498/1000 = 7.498$, prácticamente el valor exacto $m_1 m_2 = 7.5$. El pequeño desajuste (0.002) viene de dos fuentes: el ruido genuino $(m_1 e_2 + m_2 e_1)/\Delta = -0.0015$ que ya vimos en la fórmula, y el redondeo al dividir por Δ, que aporta como mucho media unidad entera — aquí, 0.494, o 0.000494 tras decodificar. Ambas fuentes son minúsculas comparadas con el propio mensaje; es precisamente ese margen el que el rescaling protege en cada multiplicación.

En el ejemplo hemos trabajado con enteros normales, sin módulo, para poder seguir la cuenta a mano — pero conviene no perder de vista que en el esquema real esa misma división por Δ es, a la vez, la que hace descender el cifrotexto un peldaño en la cadena de módulos. No son dos operaciones distintas que coincidan por casualidad: dividir el valor cifrado por Δ *es* reducir el módulo bajo el que vive de q_i a $q_{i+1} = q_i/\Delta$. Como nuestro ejemplo solo tiene una multiplicación, q_i es aquí q_0 — el módulo del cifrotexto recién cifrado, antes de multiplicar. Si nuestro cifrotexto de juguete viviera módulo $q_0 = 1000 \times 7\,500\,000$ (para dar margen de sobra al producto sin desbordarse), tras dividir por $\Delta = 1000$ pasaría a vivir módulo $q_1 = q_0/1000$ — un módulo mil veces más pequeño, porque ya no hace falta alojar los dígitos extra que había añadido la multiplicación. Esta es la razón por la que cada rescaling «consume un nivel»: el módulo se hace cada vez más pequeño y, tras L divisiones, no queda margen para seguir dividiendo.

Contrastemos con multiplicar por una constante en claro, digamos $k = 4$, en vez de por m_2. El cifrotexto sigue siendo $\Delta m_1 + e_1 = 2502$; multiplicarlo por k da $k(\Delta m_1 + e_1) = 4 \times 2502 = 10008$. Comparado con el valor ideal $\Delta(k m_1) = 1000 \times 4 \times 2.5 = 10000$, la diferencia es 8 — exactamente $k e_1 = 4 \times 2 = 8$, el mismo ruido escalado por k que vimos en «Multiplicación por constante». La diferencia clave está en la escala: 10008 ya vive en la escala estándar $\Delta = 1000$ (al descifrar, $10008/1000 = 10.008 \approx k m_1 = 10$), porque k es un número suelto sin su propio factor Δ — a diferencia de m_2, que sí lo llevaba dentro del segundo cifrotexto. No hay ningún Δ^2 que corregir y, por tanto, nada que rescalar: multiplicar por una constante en claro no consume ningún eslabón de la cadena de módulos, solo hace crecer el ruido por el factor k.

En concreto, un esquema *leveled* (por niveles) formaliza esta idea con una cadena de módulos decrecientes $q_0 > q_1 > \cdots > q_L$, tal como acabamos de

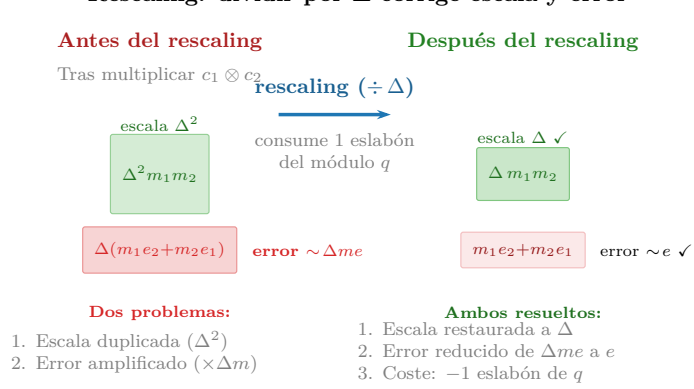


Figura 5.6: Rescaling: dividir por Δ corrige simultáneamente la escala duplicada y el error amplificado, a cambio de consumir un eslabón de la cadena de módulos.

ilustrar con el ejemplo numérico: cada rescaling divide el cifrotexto por Δ y reduce el módulo al siguiente de la cadena, $q_i \to q_{i+1} = q_i/\Delta$. Cuando se agotan los eslabones, no quedan más niveles para multiplicar.

Esto es exactamente lo que configura `coeff_mod_bit_sizes` en el código que utiliza TenSEAL. Veámoslo con nuestro ejemplo:

```
coeff_mod_bit_sizes = [60, 40, 40, 60]
```

- El **último valor** (60 bits) es un módulo auxiliar reservado para la relinearización que vimos antes — el paso que convierte el triplete (d_0, d_1, d_2) en un par. La librería necesita un módulo extra de gran tamaño para ejecutar esa operación con precisión, pero no cuenta como nivel de multiplicación.
- El **primer valor** (60 bits) es el módulo base q_0, el único que sobrevive hasta el final de la cadena, cuando se agotan los eslabones intermedios. Su tamaño en bits, comparado con la escala Δ (`global_scale`, que se fija por separado), determina cuánto margen de precisión entera queda disponible en ese último eslabón: con q_0 de 60 bits y $\Delta = 2^{40}$, quedan $60 - 40 = 20$ bits — algo más de un millón — para la parte entera del resultado antes del punto decimal. Si el cómputo llega a ese último eslabón con un valor que supera ese margen, CKKS no lo detecta: el resultado se desborda en silencio, como le pasó a Daniel con la versión ingenua de la varianza en la Receta 2.

- Los **valores intermedios** (40, 40) son los eslabones de la cadena de *rescaling*: cada uno representa un nivel de multiplicación disponible. Con dos valores de 40 bits, el esquema soporta hasta dos multiplicaciones encadenadas.

Aunque la lista tiene cuatro valores, un cifrotexto recén cifrado nunca llega a contener los cuatro: el módulo de relinearización solo existe al nivel de las claves, y la librería lo descarta antes de generar el primer cifrotexto de datos. Un cifrotexto fresco arranca entonces con tres módulos — el primer valor más los dos eslabones de 40 bits —, no cuatro. Cada multiplicación consume uno de esos dos eslabones intermedios; tras la segunda, no queda ningún tercer eslabón que consumir — ni otro de 40 bits (ya se gastaron los dos) ni el último 60 de la lista, que nunca estuvo ahí para empezar. Lo único que queda es el primer valor: el límite de precisión que acabamos de ver.

El número 40 no es arbitrario: coincide con `global_scale = 2**40` que fijamos al crear el contexto. Cada *rescaling* divide por la escala (2^{40}), lo que requiere consumir exactamente un módulo de 40 bits de la cadena.

¿Y por qué 40 bits y no, por ejemplo, un Δ tan grande como el propio q? Porque Δ fija el tamaño del eslabón que cada multiplicación consume: cuanto mayor sea Δ, más dígitos decimales de precisión obtenemos (unos 12 con 2^{40}, comparable a un *double*), pero también más presupuesto de módulo se agota en cada rescaling. Si Δ fuera comparable a q, el primer rescaling consumiría de golpe casi todo el presupuesto disponible, dejando margen para una sola multiplicación por mucho que aumentemos N — justo lo que la cadena de niveles busca evitar. Un valor moderado, típicamente entre 30 y 60 bits, equilibra precisión decimal con el número de multiplicaciones encadenadas disponibles.

Nótese que no es Δ quien debe acercarse a q_0, sino Δ^L — el producto acumulado de las L divisiones, una por cada rescaling. Con `[60, 40, 40, 60]` y $L = 2$, $q_0 \approx 2^{200}$ mientras que $\Delta^L = (2^{40})^2 = 2^{80}$: solo un fragmento del presupuesto total: el resto son los dos eslabones de 60 bits reservados para la codificación y la relinearización, que no se consumen por rescaling. Si Δ mismo fuera del tamaño de q_0, no habría margen para encadenar una segunda división — sería como levantar un muro con un único ladrillo del tamaño del muro entero.

¿Cómo decide un ingeniero estos valores? La regla práctica es: cuenta cuántas multiplicaciones encadenadas necesita tu circuito (llamémoslo L) y configura L valores intermedios de 40 bits (para unos 12 dígitos de precisión decimal), más un 60 al principio y otro al final. Para calcular una media

(suma + división por constante) basta con una multiplicación, así que $L = 1$ sería suficiente. Nuestro ejemplo usa $L = 2$ para tener margen. Si necesitaras, por ejemplo, calcular un polinomio de grado 4, usarías `[60, 40, 40, 40, 40, 60]` — cuatro niveles intermedios. Más niveles implican un módulo q total más grande, lo que a su vez requiere un `poly_modulus_degree` mayor para mantener la seguridad.

¿Por qué se escoge un valor u otro de q, el módulo total, Δ, el factor de rescaling, y N, la dimensión del retículo? Recordemos que la seguridad de RLWE descansa en que el error e inyectado al cifrar debe ser lo bastante pequeño para ser eliminado al descifrar, pero lo bastante grande para «ocultar» el mensaje a un atacante. Un atacante que intenta romper el esquema debe distinguir un cifrotexto RLWE — $b = a \cdot s + e + m$ — de un par de polinomios puramente aleatorios.

Cuanto mayor sea q relativo al error e, más fácil le resulta distinguir el cifrotexto del ruido puro: el error ocupa una fracción menor del espacio de posibles valores módulo q, y el cifrotexto se parece cada vez más a un producto exacto $a \cdot s$ sin ruido, que sería trivial de invertir. Aumentar N — la dimensión del retículo subyacente — compensa ese debilitamiento, porque hace el problema exponencialmente más difícil por la misma razón que vimos al introducir RLWE: los mejores algoritmos de reducción de retículos tienen coste exponencial en la dimensión.

Como regla práctica, los *bits de seguridad* — el exponente λ tal que el atacante necesita $\approx 2^\lambda$ operaciones para romper el esquema por fuerza bruta — son aproximadamente proporcionales a $N / \log_2 q$. De ahí una consecuencia directa: si duplicas el tamaño de q añadiendo niveles a la cadena, necesitas aumentar N en la misma proporción para que el coste de ataque siga siendo exponencial.

Por ejemplo, con 4 niveles la cadena `[60, 40, 40, 40, 40, 60]` suma 280 bits de módulo total, frente a los 200 bits de nuestro ejemplo original. Para mantener 128 bits de seguridad con ese módulo mayor, típicamente necesitarás $N = 16384$ en vez de 8192.

Pero si el presupuesto es finito, algo no cuadra: llamamos al esquema *completamente* homomórfico, y sin embargo solo permite un número fijo de multiplicaciones — no muy diferente, en principio, de los esquemas parciales que mencionamos al inicio del capítulo.

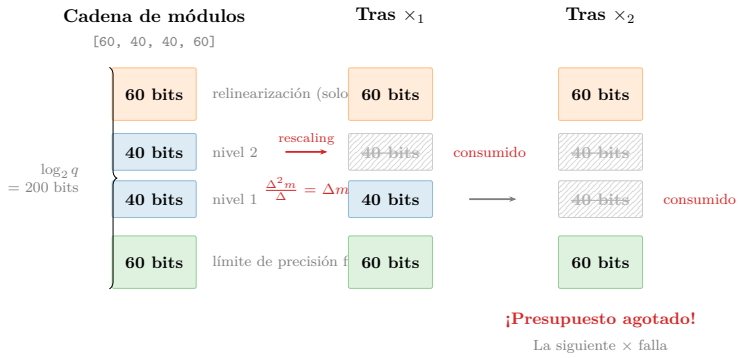


Figura 5.7: Cadena de módulos y consumo de niveles. Izquierda: la cadena inicial `[60, 40, 40, 60]` con sus cuatro valores — el de 60 bits superior es el módulo auxiliar de relinearización, que solo existe al nivel de las claves y nunca forma parte del cifrotexto de datos; los dos de 40 bits son los eslabones de multiplicación disponibles; el de 60 bits inferior es el módulo base que sobrevive hasta el final, fijando el límite de precisión. Centro: tras la primera multiplicación, el rescaling divide por Δ y consume el eslabón «nivel 2», dejando uno disponible. Derecha: tras la segunda multiplicación se consume también «nivel 1» y el presupuesto queda agotado — una tercera multiplicación ya no tendría eslabón que consumir y fallaría.

Preguntas de repaso

5.10. ¿Por qué el *rescaling* es necesario después de cada multiplicación en CKKS? ¿Qué ocurre con la escala del mensaje si no se aplica?

5.11. Para un circuito que calcula $(a \cdot b) \cdot c$, ¿cuántos niveles se necesitan? Compáralo con $a \cdot b + c \cdot d$. ¿Qué determina el número de niveles necesarios: la cantidad total de multiplicaciones o la profundidad del circuito?

5.5 Bootstrapping

Hemos recorrido un camino que va de los retículos al álgebra y de allí a los polinomios en RLWE, del cifrado a las operaciones homomórficas, y del crecimiento del ruido a la cadena de módulos que lo gestiona. Pero todo lo descrito hasta aquí define un esquema *leveled* — capaz de un número prefijado de multiplicaciones. Para merecer el nombre de completamente homomórfico (la F del acrónimo FHE), necesitamos una forma de restaurar el presupuesto agotado. Esa es la contribución clave de Craig Gentry en 2009, que ya mencionamos en la historia del FHE: el *bootstrapping* — la misma técnica que Daniel aplicó en la Receta 5 para rescatar el cálculo que había dejado a Elena con el «círculo de la muerte» en pantalla.

Supongamos que tras L multiplicaciones hemos agotado la cadena de módulos: cada *rescaling* ha dividido el módulo por Δ, dejando el cifrotexto en el último eslabón $q_L = q_0/\Delta^L$, sin niveles para seguir operando. El error es pequeño gracias al *rescaling*, pero no hay más eslabones que consumir.

La idea del bootstrapping es contraintuitiva: tomamos ese cifrotexto «agotado» y lo desciframos *homomórficamente*. Es decir, el propio circuito de descifrado $(b - a \cdot s)$ se evalúa como una operación sobre cifrotextos. Para ello, la clave secreta s se cifra de antemano bajo la propia clave s y se publica como *clave de bootstrapping* (CB). Retomando la analogía de Gentry que vimos al inicio del capítulo, el sistema crea una nueva «caja de guantes» dentro de la caja de guantes agotada, que la reemplaza: el cifrotexto agotado entra, se descifra internamente y se vuelve a cifrar, todo sin salir nunca del dominio cifrado. El resultado es un cifrotexto *fresco* — con el módulo restaurado a q_0 y el presupuesto de niveles completo — que contiene el mismo mensaje.

Veamos paso a paso cómo funciona. Partimos del cifrotexto agotado $c = (a, b)$ en el último nivel L, con módulo q_L. Sabemos que:

$$b - a \cdot s \equiv \Delta\, m + e \pmod{q_L}$$

donde e es pequeño gracias al rescaling acumulado. Los valores a y b son conocidos — son el propio cifrotexto, que el servidor tiene en sus manos. Lo que no conoce es s.

Antes de empezar cualquier cómputo, el propietario de la clave ha publicado una *clave de bootstrapping* (CB): un cifrado fresco de la clave secreta s bajo la propia clave s, pero con el módulo completo q_0. Siguiendo la misma notación de cifrado que definimos antes — $\mathrm{Enc}(m) = (a,\ b = a \cdot s + e + m)$ — la clave de bootstrapping es:

$$\mathrm{CB} = \mathrm{Enc}(s) = (a_{\mathrm{cb}},\ b_{\mathrm{cb}} = a_{\mathrm{cb}} \cdot s + e_{\mathrm{cb}} + s)$$

Es exactamente un cifrado normal, solo que el «mensaje» cifrado es la propia clave s. (Cifrar la clave bajo sí misma requiere una hipótesis llamada *seguridad circular*; todos los esquemas FHE actuales la asumen.) La diferencia crucial es que este cifrado se generó con el módulo completo q_0, así que tiene todo el presupuesto de niveles intacto.

Ahora el servidor puede evaluar el circuito de descifrado homomórficamente. Veamos cómo, paso a paso. Recordemos que a y b — las componentes del cifrotexto agotado — son valores concretos, polinomios que el servidor tiene en sus manos. Toda la información del mensaje m y del error acumulado e está codificada *dentro* de esos valores, en la relación $b - a \cdot s \equiv \Delta m + e \pmod{q_L}$. Los polinomios a y b en sí mismos son solo números grandes; el error no «vive» en a o en b por separado, sino en cómo se combinan con la clave s.

Pero hay un detalle crucial en ese « $\pmod{q_L}$». Lo que significa es que el valor *entero* de $b - a \cdot s$ — sin reducir módulo nada — no es $\Delta m + e$, sino:

$$b - a \cdot s = \Delta\, m + e + k \cdot q_L$$

para algún entero k. Al reducir módulo q_L, el término $k \cdot q_L$ desaparece y solo queda el residuo $\Delta m + e$. Cuando el cifrotexto operaba bajo el módulo q_L, eso no importaba — la reducción se aplicaba implícitamente. Pero ahora que vamos a pasar a un módulo mayor, ese $k \cdot q_L$ se convierte en un problema. Veámoslo paso a paso:

Paso 1: descifrado homomórfico. El servidor usa a y b como constantes conocidas para operar sobre la CB. Primero, multiplica la CB (que es un

cifrotexto que contiene s) por la constante a. Ya vimos que multiplicar un cifrotexto por una constante preserva el mensaje: si CB cifra s, entonces $a \cdot \text{CB}$ cifra $a \cdot s$. Esta operación no consume nivel —no es una multiplicación entre dos cifrotextos, así que no hace falta relinearización—, pero sí dispara el ruido: como vimos en la multiplicación por constante, el error se escala por el propio multiplicador, y aquí el multiplicador no es una constante pequeña, sino a, un polinomio con coeficientes tan grandes como el propio módulo q_L. El ruido de la CB se multiplica por algo del orden de q_L, no por un factor modesto — y es justo por eso por lo que la CB se cifra bajo el módulo completo q_0: necesita ese margen extra para absorber un salto de ruido de esa magnitud sin desbordarse. Después, niega el resultado y suma la constante b — negar es multiplicar por la constante -1, y sumar una constante a un cifrotexto simplemente suma esa constante al mensaje cifrado; ninguna de las dos altera el ruido. Escribimos esa combinación de forma compacta como $b - a \cdot \text{CB}$, donde a y b son los componentes conocidos del cifrotexto agotado (no confundir con $a_{\text{cb}}, b_{\text{cb}}$, los componentes internos de CB). Como Enc siempre produce un par, esa notación compacta abrevia el par real, obtenido negando y sumando componente a componente:

$$b - a \cdot \text{CB} = (-a \cdot a_{\text{cb}},\ \ b - a \cdot b_{\text{cb}}) = \text{Enc}(b - a \cdot s) = \text{Enc}(\Delta\, m + e + k \cdot q_L)$$

Este nuevo cifrotexto fue generado a partir de la CB, que tiene el módulo completo q_0. Por tanto tiene niveles de sobra para seguir operando. Pero contiene $\Delta m + e + k \cdot q_L$ — el mensaje que queremos *más* un término espurio $k \cdot q_L$ que hay que eliminar.

Paso 2: eliminación del término $k \cdot q_L$ (reducción modular homomórfica). Aquí reside el coste real del bootstrapping. Necesitamos calcular $(b - a \cdot s) \bmod q_L$ homomórficamente para quedarnos solo con $\Delta m + e$. Pero la reducción modular no es una operación polinómica — no se puede expresar como una secuencia de sumas y multiplicaciones. Para evaluarla homomórficamente hay que *aproximarla* mediante un polinomio de grado alto, lo que consume múltiples niveles de multiplicación — típicamente entre 10 y 20, según la precisión deseada.

Solo después de este segundo paso obtenemos un cifrotexto fresco bajo q_0 que contiene $\Delta m + e$ — el mensaje limpio, sin el término $k \cdot q_L$, y con el presupuesto de niveles parcialmente restaurado.

Por eso el bootstrapping es caro: el paso 1 ($b - a \cdot \text{CB}$) es trivial, pero el paso 2 (la reducción modular) es un cálculo pesado que consume buena parte de los niveles recién restaurados.

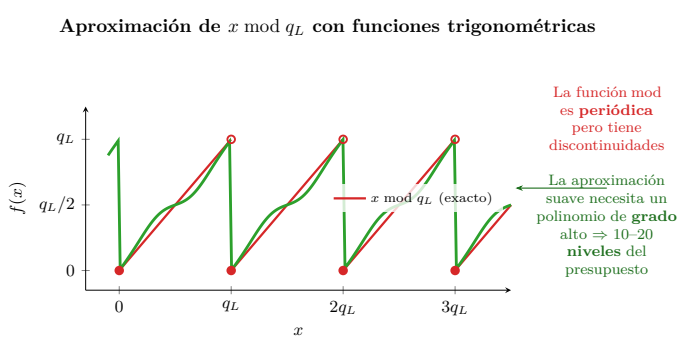


Figura 5.8: Aproximación de la reducción modular. La función «x módulo q_L» es periódica pero discontinua — no se puede expresar como una secuencia de sumas y multiplicaciones. Para evaluarla homomórficamente se aproxima mediante un polinomio de grado alto, que consume entre 10 y 20 niveles multiplicativos.

Es como vaciar el depósito de combustible de un avión hasta la reserva y, en vez de aterrizar, usar la propia reserva para accionar una bomba que rellena el tanque principal desde un depósito auxiliar que llevabas a bordo.

En la práctica, un bootstrap tarda del orden de un segundo — frente a los milisegundos de una operación normal — y el cifrotexto resultante no recupera *todos* los niveles, porque la propia evaluación del circuito de descifrado consume parte del presupuesto restaurado. Por esta razón, el enfoque preferido es diseñar el cálculo para que quepa dentro del presupuesto de niveles (*leveled FHE*) y recurrir al bootstrapping solo cuando el circuito es demasiado profundo para dimensionarlo de antemano.

En la práctica, la situación en CKKS es algo más compleja: los datos no se almacenan directamente como coeficientes del polinomio, sino codificados mediante una transformada — lo que añade pasos adicionales (*CoeffsToSlots* y *SlotsToCoeffs*) al circuito de bootstrapping. Veremos esa codificación en detalle en la siguiente sección.

Con o sin bootstrapping, cada operación homomórfica tarda milisegundos — un coste que se paga por *cifrotexto*. Si cada cifrotexto contiene un solo dato, el rendimiento resulta prohibitivo para cualquier aplicación real.



Figura 5.9: Bootstrapping en dos pasos. El Paso 1 es barato: multiplica la clave CB por constantes conocidas. El Paso 2 es el cuello de botella (aproximadamente 1 segundo), porque aproxima la reducción modular mediante un polinomio de alto grado.

Preguntas de repaso

5.12. ¿Por qué el bootstrapping necesita cifrar la clave secreta bajo sí misma? ¿Qué hipótesis de seguridad requiere esto?

5.13. Si el bootstrapping es tan caro, ¿por qué no se dimensionan siempre los parámetros para evitarlo? ¿Cuándo es inevitable?

5.6 Packing y SIMD

Si cada operación homomórfica cuesta milisegundos independientemente de cuántos datos contenga el cifrotexto, la clave para un rendimiento aceptable está en empaquetar el máximo número de datos en cada uno. Gracias a una técnica llamada *batching* o *packing*, un solo cifrotexto puede contener miles de valores independientes, y una sola operación sobre el cifrotexto los procesa *todos en paralelo*. El packing es lo que convierte al FHE de una curiosidad teórica en una herramienta con rendimiento aceptable — es, de hecho, la razón por la que el código de Daniel procesa los miles de salarios de Elena casi tan rápido como procesaría uno solo. Pero ¿cómo caben miles de números dentro de un solo polinomio? La respuesta conecta la estructura algebraica del anillo R_q con una idea que ya conocemos: la transformada de Fourier.

Para entender por qué el packing es necesario, consideremos un ejemplo sencillo: ¿cómo podríamos codificar los cinco salarios $[3200, 4100, 2800, 5500, 3900]$ en uno o varios polinomios? La forma más fácil sería codificar cada valor como un polinomio de grado cero, es decir, una constante — $m(x) = 3200$, $m(x) = 4100$, etc. — y cifrar cada uno por separado. El problema es que

cada cifrotexto es un par de polinomios de grado $N - 1 = 8191$, y por lo tanto con $N = 8192$ coeficientes enteros módulo q: en nuestro ejemplo, unos 400 KB por cifrotexto. De esos 8192 coeficientes, solo el término constante llevaría información útil; los otros 8191 serían ceros. Para cinco salarios necesitaríamos cinco cifrotextos independientes — 2 MB en total — y cinco operaciones separadas para cualquier cálculo. Un polinomio de 8192 coeficientes transportando un único número es un desperdicio absurdo, como fletar un camión de 18 ruedas para entregar una carta.

La estructura algebraica de los polinomios en el anillo R_q ofrece algo mucho mejor: empaquetar los cinco valores — o hasta $N/2 = 4096$ — en un *único* polinomio, cuyo resultado no solo ocupa menos espacio, sino que además una sola operación procesa todos los valores a la vez. Los datos no se colocan en los coeficientes del polinomio: se codifican en sus *evaluaciones* en puntos específicos del plano complejo que de hecho son las raíces de $x^N + 1 = 0$, el módulo del anillo.

De coeficientes a slots. Recordemos que un cifrotexto vive en el anillo R_q que definimos antes como el conjunto de polinomios módulo $x^N + 1 = 0$: un polinomio $m(x)$ de grado menor que N con coeficientes enteros módulo q. A primera vista, este polinomio tiene N coeficientes y punto — no hay «posiciones» independientes donde colocar datos. Pero ocurre algo interesante cuando evaluamos ese polinomio en puntos concretos — una operación que en álgebra se llama *canonical embedding* (inmersión canónica).

Fijémonos en que el polinomio $x^N + 1$ tiene N raíces complejas: $\omega_k = e^{i\pi(2k+1)/N}$ para $k = 0, 1, \ldots, N-1$. Son puntos equidistribuidos en el círculo unitario del plano complejo — exactamente los mismos tipos de raíces que aparecen en la FFT. Evaluar $m(x)$ en esas N raíces produce N números complejos. Pero como los coeficientes de $m(x)$ son reales (enteros), las evaluaciones en raíces conjugadas son también conjugadas: $m(\bar{\omega}_k) = \overline{m(\omega_k)}$. Las raíces se agrupan en $N/2$ pares conjugados, y cada par aporta un solo valor complejo independiente. Los $N/2$ valores complejos resultantes son los *slots*.

Cada slot tiene una parte real y una parte imaginaria, así que en principio se podrían empaquetar hasta N números reales — $N/2$ en las partes reales y $N/2$ en las imaginarias. Pero usar de forma independiente ambas partes de cada slot forzaría a que los coeficientes del polinomio, tras la transformada inversa, fueran números complejos en lugar de enteros — y el anillo R_q solo admite coeficientes enteros módulo q. Por eso CKKS empaqueta $N_{\text{slot}} = N/2$ valores reales (uno por slot, usando solo la parte real): mezclar partes reales e imaginarias complicaría la gestión de errores y duplicaría el almacenamiento necesario para representar coeficientes complejos. Con

`poly_modulus_degree = 8192`, tenemos $N_{\text{slot}} = 4096$ slots disponibles.

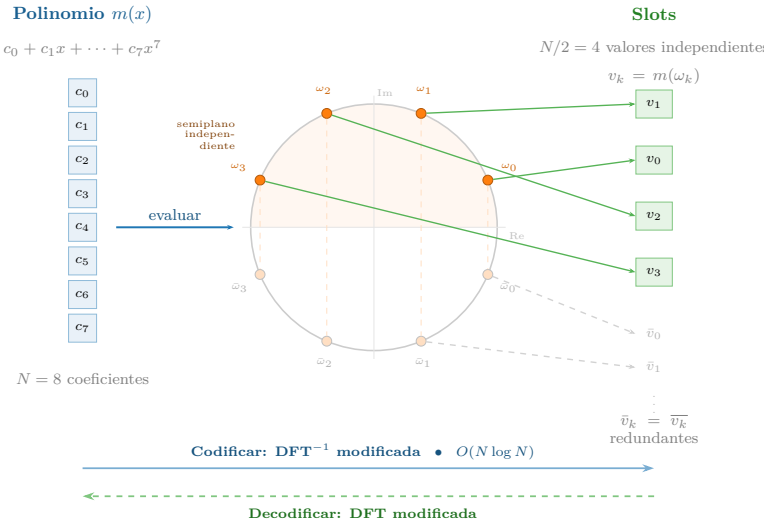


Figura 5.10: Inmersión canónica. Los coeficientes del polinomio $m(x)$ no son los datos: los slots son las evaluaciones de $m(x)$ en las N raíces complejas de $x^N + 1$. Las raíces conjugadas producen valores redundantes, dejando $N/2$ slots independientes. Codificar y decodificar equivale a una DFT modificada (inversa y directa, respectivamente).

Codificar un vector de datos — por ejemplo, los cinco salarios $[3200, 4100, 2800, 5500, 3900]$ — consiste en encontrar el polinomio $m(x)$ de grado $N-1$ cuyas evaluaciones en las $N/2$ raíces coincidan con esos valores. Es exactamente una transformada inversa de fourier: dados los valores en el «dominio de frecuencias» (los slots), se calcula la señal en el «dominio del tiempo» (los coeficientes del polinomio). La librería lo hace internamente con una variante de la inversa de la DFT, que cuesta $O(N \log N)$ — la misma complejidad que la NTT que ya usamos para multiplicar polinomios. Los cinco salarios se colocan en los primeros 5 de los 4096 slots; los restantes 4091 quedan a cero.

¿Por qué funciona el paralelismo? Esta transformación no solo mejora el empaquetamiento: además paraleliza las sumas y multiplicaciones sobre el cifrotexto. La clave es que las operaciones en el anillo R_q — suma y multiplicación de los polinomios *en claro* $m(x)$, antes de cualquier cifrado — se traducen *automáticamente* en operaciones elemento a elemento sobre los

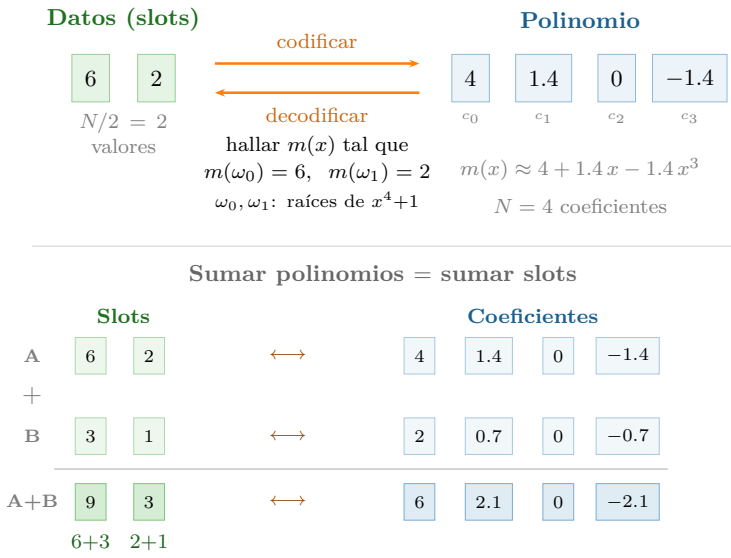


Figura 5.11: Codificación por slots ($N = 4$). Arriba: dos datos $[6, 2]$ se codifican como un polinomio de 4 coeficientes hallando $m(x)$ tal que $m(\omega_0) = 6$ y $m(\omega_1) = 2$. Abajo: sumar dos polinomios coeficiente a coeficiente equivale a sumar los slots dato a dato — la base del paralelismo SIMD.

slots. Si $m_1(x)$ codifica el vector de evaluaciones $(v_1, v_2, \dots, v_{N/2})$ y $m_2(x)$ codifica $(w_1, w_2, \dots, w_{N/2})$, entonces:

- $m_1(x) + m_2(x)$ codifica $(v_1 + w_1,\ v_2 + w_2,\ \dots,\ v_{N/2} + w_{N/2})$
- $m_1(x) \cdot m_2(x)$ codifica $(v_1 \cdot w_1,\ v_2 \cdot w_2,\ \dots,\ v_{N/2} \cdot w_{N/2})$

Nótese que este producto $m_1(x) \cdot m_2(x)$ es la multiplicación ordinaria de los dos polinomios en claro dentro del anillo — no la operación $c_1 \otimes c_2$ entre cifrotextos que vimos en la sección anterior, con su triplete (d_0, d_1, d_2) y su relinearización. Es precisamente este hecho algebraico sobre los polinomios en claro el que hace que la multiplicación homomórfica funcione slot a slot: como $\mathrm{Dec}(c_1 \otimes c_2) \approx m_1(x) \cdot m_2(x)$, y este producto en claro coincide con el producto elemento a elemento de los datos, descifrar el resultado de $c_1 \otimes c_2$ produce exactamente $(v_1 \cdot w_1, \dots, v_{N/2} \cdot w_{N/2})$.

Esto es una consecuencia directa de la estructura algebraica y de la transformación de valores a slots, las evaluaciones del polinomio en las raíces de $x^N + 1$: evaluar un polinomio en un punto es un homomorfismo de anillos — si $f(\omega) = v$ y $g(\omega) = w$, entonces $(f + g)(\omega) = v + w$ y $(f \cdot g)(\omega) = v \cdot w$. Una sola operación sobre el polinomio equivale a $N/2$ operaciones independientes sobre los datos — la misma idea que las instrucciones SIMD (*Single Instruction, Multiple Data*) de un procesador moderno.

En una CPU con instrucciones AVX-512, un registro de 512 bits almacena 8 *doubles* de 64 bits, y una sola instrucción `VADDPD` (*Vector Add Packed Double-Precision Floating-Point*) suma los 8 pares. En FHE, el cifrotexto hace de «registro» y almacena hasta $N_{\text{slot}} = N/2 = 4096$ valores; una sola suma homomórfica los procesa todos a la vez. La estructura es la misma; lo que cambia es la escala del coste: la instrucción AVX tarda del orden de un nanosegundo, mientras que la operación homomórfica tarda del orden de milisegundos — el extremo superior de la brecha de 10^4–10^6 que vimos en Capítulo 4. Lo decisivo es que ese coste se paga *por cifrotexto*, no por dato: no depende de cuántos slots estén ocupados, porque se ejecuta *una* única operación polinómica sobre el anillo, lleve esta 5 valores o 4096. Repartido entre los 4096 slots, el coste por valor baja unos tres órdenes de magnitud — y es esa amortización, no una operación intrínsecamente más rápida, la que hace viable el FHE.

Economía del packing. Sin packing, cada cifrotexto almacena un solo valor y ocupa ~400 KB. Cifrar 4096 valores requeriría 4096 cifrotextos — 1.6 GB — y 4096 operaciones para sumarlos. Con packing, los 4096 valores caben en un solo cifrotexto de 400 KB, y una sola suma los procesa todos a la vez. El coste amortizado baja de 400 KB a ~100 bytes por valor, y el número de operaciones se reduce en un factor $N/2$. Es esta reducción

la que permite que aplicaciones como la regresión cifrada (Receta 3) o la búsqueda semántica (Receta 6) sean viables en la práctica.

En nuestro código, cuando escribimos `ts.ckks_vector(contexto, salarios)`, la librería ejecuta internamente esa codificación: transformando el vector Python `[3200, 4100, 2800, 5500, 3900]` en un polinomio de grado 8191 cuyas evaluaciones en las raíces de $x^{8192} + 1$ reproducen esos cinco valores en los primeros slots. El programador no necesita pensar en raíces complejas ni en transformadas — solo en vectores y operaciones aritméticas.

Ahora bien, el packing tiene una limitación fundamental: solo permite operaciones *slot a slot*. Sumar o multiplicar dos vectores cifrados funciona directamente — cada slot se procesa de forma independiente. Pero ¿qué ocurre cuando necesitamos combinar valores de distintos slots? Por ejemplo, para calcular `salarios_cifrados.sum()` hay que sumar los cinco valores que están en slots separados. Eso requiere *mover datos entre slots* — una operación que no se consigue con sumas ni multiplicaciones de polinomios, y que necesita un mecanismo completamente distinto: las rotaciones.

Preguntas de repaso

5.14. ¿Por qué el número de slots es $N/2$ y no N? ¿Qué propiedad de las raíces de $x^N + 1$ lo explica?

5.15. En nuestro ejemplo, `ts.ckks_vector(contexto, salarios)` coloca 5 valores en 4096 slots. ¿Qué contienen los 4091 slots restantes? ¿Afecta eso al coste de las operaciones?

5.16. En CKKS, el bootstrapping necesita aplicar la reducción modular sobre cada *slot* de forma independiente, pero el descifrado homomórfico produce el polinomio en su representación de coeficientes. ¿Por qué es necesario un paso adicional (*CoeffsToSlots*) antes de la reducción modular? ¿Qué tipo de operación es y qué coste tiene?

5.7 Rotaciones

Las operaciones elemento a elemento — sumas y multiplicaciones slot por slot — son la base del paralelismo SIMD en FHE. Pero muchos cálculos útiles necesitan *combinar* valores que están en distintos slots del mismo cifrotexto. Calcular la media de un vector cifrado requiere sumar todos sus elementos; una multiplicación de matrices requiere productos escalares entre filas y columnas; una convolución requiere desplazar y acumular. Todas

estas operaciones exigen mover datos entre posiciones del vector — exactamente lo que hace `salarios_cifrados.sum()`, la línea que Daniel escribió sin pensarlo dos veces en la Receta 1. Implementadas de forma ingenua, su coste sería proporcional al número de elementos del vector; las *rotaciones* permiten reducirlo de lineal a logarítmico.

Antes de ver cómo se implementan las rotaciones en el esquema — que involucra automorfismos sobre polinomios y claves adicionales —, conviene entender la idea a nivel de vectores, sin preocuparnos todavía por la maquinaria criptográfica subyacente.

Desplazamiento cíclico. Una rotación por k posiciones desplaza todos los slots del vector k posiciones hacia la izquierda, de forma cíclica: el valor que estaba en el slot i pasa al slot $i - k$, y los que «salen» por un extremo reaparecen por el otro. Con cuatro slots y una rotación unitaria como ejemplo:

$$\mathrm{Rot}([a, b, c, d],\ 1) = [b, c, d, a]$$

El dato a del slot 0 pasa al slot 3; el dato b del slot 1 pasa al slot 0, y así sucesivamente. Es un desplazamiento circular, no una pérdida de datos — todos los valores siguen en el cifrotexto, solo que en posiciones diferentes.

Suma por árbol de rotaciones. Para sumar todos los elementos de un vector cifrado, no existe una operación «sumar todos los slots» — recuerda que el esquema solo ofrece sumas y multiplicaciones de polinomios. La solución es rotar y sumar repetidamente. Ilustrémoslo con 4 slots:

$$[a, b, c, d] \xrightarrow{+\,\mathrm{Rot}(\cdot,2)} [a{+}c,\ b{+}d,\ c{+}a,\ d{+}b] \xrightarrow{+\,\mathrm{Rot}(\cdot,1)} [a{+}b{+}c{+}d,\ \ldots]$$

En el primer paso, rotamos 2 posiciones y sumamos el resultado con el original: cada slot acumula la suma de dos elementos. En el segundo paso, rotamos 1 posición y sumamos de nuevo: cada slot contiene la suma de los cuatro. En general, $\log_2(N_{\text{slot}})$ pasos bastan para sumar los N_{slot} elementos. Con $N_{\text{slot}} = 4096$, son 12 rotaciones y 12 sumas — exactamente lo que ejecuta `.sum()` internamente. La ganancia es sustancial: sin este truco, sumar los valores de N_{slot} slots requeriría $N_{\text{slot}} - 1$ operaciones individuales — una acumulación por cada slot. El árbol de rotaciones reduce ese coste de $O(N_{\text{slot}})$ a $O(\log_2 N_{\text{slot}})$: con 4096 slots, de 4095 operaciones a solo 24. La idea es la misma que convierte una búsqueda lineal en binaria — dividir el problema a la mitad en cada paso: cada iteración duplica el número de

elementos acumulados en cada slot, y tras $\log_2 N_{\text{slot}}$ pasos todos contienen la suma total.

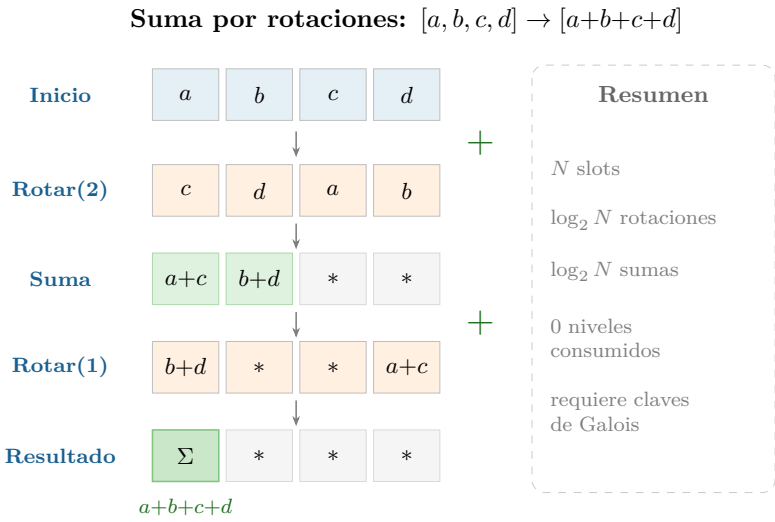


Figura 5.12: Suma por rotaciones. Con 4 slots, dos pasos de rotar-y-sumar bastan para acumular la suma total en el primer slot. El diagrama muestra la vista por slots; internamente, cada «Suma» es una adición de los N coeficientes de dos polinomios, que por linealidad de la evaluación produce la suma slot a slot. Las rotaciones requieren claves de Galois, pero no consumen niveles multiplicativos.

¿Cómo funciona una rotación sobre un polinomio? Aquí la matemática del anillo R_q revela un truco elegante. Recordemos que los slots son las evaluaciones del polinomio $m(x)$ en las raíces $\omega_0, \omega_1, \ldots, \omega_{N/2-1}$ del polinomio $x^N + 1$. Permutar los slots — mover el valor del slot i al slot j — equivale a permutar las raíces en las que se evalúa el polinomio.

Resulta que ciertas sustituciones de la variable del polinomio — llamadas *automorfismos* del anillo — hacen exactamente eso. La sustitución $x \to x^k$ (para ciertos valores impares de k) reordena las raíces de $x^N + 1$ y, por tanto, permuta los slots. Trabajemos el ejemplo del diagrama (Figura 5.13): $N = 8$, 4 slots, sustitución $x \to x^3$.

Paso 1: qué es un slot. En CKKS, el slot j es el valor que toma el polinomio $p(x)$ al evaluarlo en la raíz $\omega^{5^j \bmod 16}$, donde $\omega = e^{i\pi/8}$ es una

raíz primitiva de $x^8 + 1$. Para $N = 8$ hay 4 slots, con raíces $\omega^1, \omega^5, \omega^9, \omega^{13}$.

Paso 2: qué hace la sustitución al polinomio. Aplicar $x \to x^3$ transforma $p(x)$ en el nuevo polinomio $q(x) = p(x^3)$. El valor del slot j en q es:

$$q(\omega^{5^j}) = p\Big((\omega^{5^j})^3\Big) = p\Big(\omega^{3 \cdot 5^j \bmod 16}\Big)$$

Es decir, el slot j del nuevo polinomio toma el valor que p tenía en la raíz $\omega^{3 \cdot 5^j \bmod 16}$.

Paso 3: calcular adónde va cada slot. Computamos $3 \cdot 5^j \bmod 16$ para $j = 0, 1, 2, 3$:

Slot j	Raíz original	$3 \cdot 5^j \bmod 16$	Nueva raíz	Valor en el slot
0	ω^1	$3 \cdot 1 = 3$	$\omega^3 = \overline{\omega^{13}}$	d (slot 3 original)
1	ω^5	$3 \cdot 5 = 15$	$\omega^{15} = \overline{\omega^1}$	a (slot 0 original)
2	ω^9	$3 \cdot 9 = 27 \equiv 11$	$\omega^{11} = \overline{\omega^5}$	b (slot 1 original)
3	ω^{13}	$3 \cdot 13 = 39 \equiv 7$	$\omega^7 = \overline{\omega^9}$	c (slot 2 original)

Paso 4: por qué el conjugado da el mismo valor. En cada caso, la nueva raíz es el conjugado complejo de la raíz de otro slot. Para datos reales, $p(\bar{z}) = \overline{p(z)} = p(z)$ — el valor en el conjugado coincide con el valor original. Por tanto, el slot j del resultado contiene el valor del slot $j-1$ del original.

El efecto neto es una **rotación cíclica derecha**: $[a, b, c, d] \to [d, a, b, c]$. Eligiendo distintos valores de k impar se obtienen permutaciones distintas — cada k es una rotación diferente.

¿Por qué hacen falta claves de Galois? Aquí está el problema práctico. Aplicar $x \to x^k$ al cifrotexto $(a(x), b(x))$ produce $(a(x^k), b(x^k))$. Este nuevo par sigue cifrando el mensaje rotado, pero bajo una clave *diferente*: $s(x^k)$ en lugar de $s(x)$. Es como si al rotar los guantes de la caja de seguridad se hubiera cambiado la cerradura — el resultado es correcto, pero ya no podemos abrirlo con nuestra llave original.

Para convertir el cifrotexto de vuelta a la clave $s(x)$ se necesita un paso adicional llamado *key-switching* — conceptualmente similar a la relinearización que vimos en la multiplicación, pero convirtiendo $s(x^k)$ a $s(x)$ en

lugar de s^2 a s. Este paso requiere una clave precalculada que «traduce» entre las dos claves. Esa clave es la *clave de Galois* para la rotación por k posiciones, y necesitamos una distinta para cada valor de k que queramos usar.

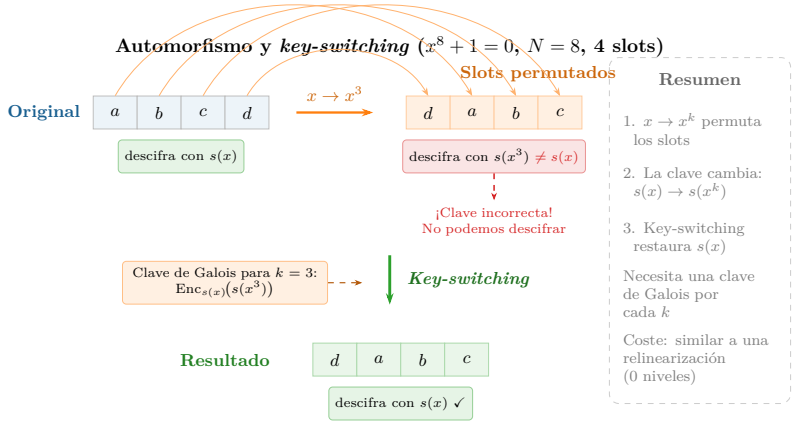


Figura 5.13: Automorfismo y key-switching (ejemplo con $x^8 + 1 = 0$, $N = 8$, 4 slots). La sustitución $x \to x^3$ permuta cíclicamente los slots: $[a, b, c, d] \to [d, a, b, c]$. Pero cambia la clave de descifrado de $s(x)$ a $s(x^3)$. El key-switching, usando una clave de Galois precalculada, restaura la clave original sin revelar el secreto.

Esto explica por qué las claves de Galois son tan voluminosas: con $N = 8192$, las $\log_2(N/2) = 12$ rotaciones necesarias para un `sum()` requieren 12 claves de Galois, cada una del tamaño de un cifrotexto (cientos de kilobytes). Si el circuito necesita rotaciones por cantidades arbitrarias, el conjunto completo puede alcanzar cientos de megabytes.

Veamos en la tabla siguiente cómo se nos han ido acumulando las claves necesarias:

Tabla 5.2: Claves en un contexto CKKS y su función.

Clave	Se genera	Habilita	¿Qué pasa sin ella?
Secreta (s)	Automáticamente al crear el contexto	Descifrar	No se pueden recuperar los resultados

Clave	Se genera	Habilita	¿Qué pasa sin ella?
Pública	Automáticamente al crear el contexto	Cifrar	No se pueden cifrar datos (solo el poseedor de s podría)
Relinearización	Automáticamente en TenSEAL	Multiplicación entre cifrotextos	El cifrotexto crece tras cada multiplicación hasta ser inmanejable
Galois	Explícitamente: `generate_galois_keys()`	Rotaciones	`sum()` y cualquier operación que combine slots falla con error

En nuestro código, `ts.context(...)` genera automáticamente la clave secreta, la pública y la de relinearización. Las claves de Galois hay que pedirlas explícitamente con `generate_galois_keys()`. ¿Por qué no se generan también automáticamente? Porque son caras: con $N = 8192$, las claves de Galois ocupan cientos de megabytes y tardan varios segundos en generarse. Si el circuito solo necesita sumas y multiplicaciones elemento a elemento — sin rotaciones —, ese coste es innecesario. Por eso la librería deja la decisión al programador.

Si Daniel olvidara esa línea e intentara ejecutar `salarios_cifrados.sum()` — que internamente necesita rotar y acumular los slots del vector —, obtendría un error en tiempo de ejecución. Las sumas y multiplicaciones elemento a elemento funcionarían sin problema, pero cualquier operación que combine slots distintos del vector fallaría.

Coste de las rotaciones. Las rotaciones no consumen niveles multiplicativos — son operaciones lineales, como las sumas. Pero no son gratis: cada rotación añade una pequeña cantidad de ruido (por el key-switching) y tiene un coste computacional similar al de una multiplicación, porque el paso de key-switching involucra multiplicaciones de polinomios. En un circuito que combine packing con rotaciones, el diseñador debe contabilizar tanto la profundidad multiplicativa como el número de rotaciones para dimensionar correctamente los parámetros.

¿Qué operaciones se benefician? Las tres primitivas del modelo SIMD — suma slot a slot, multiplicación slot a slot y rotación — cubren un catálogo bien definido de operaciones eficientes. Las *operaciones elementales* (sumar dos vectores cifrados, multiplicarlos componente a componente, escalar por una constante) son directas: una sola operación polinómica sobre todos los slots en paralelo, sin importar cuántos haya. Las *reducciones* — sumar todos los elementos, calcular una media, obtener una norma — usan

el árbol de rotaciones con coste $O(\log N_{\text{slot}})$ en lugar del $O(N_{\text{slot}})$ que costaría operar slot a slot. Los *productos escalares* combinan ambas primitivas: una multiplicación SIMD (slot a slot) seguida de una reducción por rotaciones. Las *multiplicaciones matriz-vector* se descomponen en productos escalares de cada fila con el vector. Y las *convoluciones* de redes neuronales se implementan como secuencias de rotaciones y acumulaciones. El patrón general es: cualquier cálculo que se exprese como combinaciones de sumas, multiplicaciones y desplazamientos sobre vectores se traduce eficientemente a FHE con packing y rotaciones. Lo que queda fuera son las operaciones que dependen del *valor* de los datos — comparaciones, condicionales, ordenaciones — porque el esquema no permite inspeccionar un cifrotexto para decidir qué rama tomar.

En resumen: el packing reduce el coste *por dato* cuando las operaciones son paralelas e independientes, y las rotaciones permiten combinar datos de distintos slots cuando el circuito lo requiere. Juntos, transforman la economía del FHE — el coste amortizado baja varios órdenes de magnitud y operaciones complejas como medias, productos escalares y regresiones se vuelven viables. El precio es la complejidad adicional de las claves de Galois y el diseño cuidadoso de los patrones de rotación. Este mecanismo de codificación — polinomio → evaluación en raíces → slots — también explica las transformadas *CoeffsToSlots* y *SlotsToCoeffs* que vimos en el bootstrapping: son exactamente los pasos de pasar de coeficientes del polinomio a la representación por slots y viceversa. Todo lo anterior — retículos, RLWE, cifrado, ruido, bootstrapping, packing y rotaciones — es la maquinaria de CKKS, el esquema que usamos en nuestro código. Pero esta no es la única forma de construir un esquema FHE.

Preguntas de repaso

5.17. Para sumar todos los elementos de un vector cifrado con $N_{\text{slot}} = 4096$ slots, ¿cuántas rotaciones necesita el árbol de sumas? ¿Cuántas sumas homomórficas?

5.18. ¿Por qué cada posición de rotación necesita su propia clave de Galois? ¿Qué ocurre con la clave de cifrado al aplicar la sustitución $x \to x^k$?

5.19. Pídele a un LLM: *«¿Cuál es la diferencia entre rotación de slots en FHE y una rotación (shift circular) en un registro SIMD de CPU? ¿En qué se parecen y en qué difieren?»*

5.8 Otros esquemas de encriptado

Todo lo que hemos visto describe el esquema CKKS, pero no es el único esquema FHE. Existen cuatro esquemas principales, y elegir entre ellos es una decisión de ingeniería que depende del tipo de datos, las operaciones necesarias y los compromisos aceptables.

La familia RLWE: BFV, BGV y CKKS. Los tres comparten los mismos cimientos: el anillo RLWE, la cadena de módulos, el packing SIMD y las rotaciones por automorfismos. La diferencia está en cómo codifican el mensaje antes de cifrarlo.

BFV y BGV trabajan con *enteros exactos*: el descifrado recupera el valor sin ningún error, mediante redondeo exacto (BFV) o reducción del módulo del mensaje (BGV). Eso los hace ideales para conteos, consultas sobre bases de datos cifradas o votación electrónica — escenarios donde un error de ± 1 sería inaceptable. La contrapartida es que no representan reales directamente: cualquier cálculo con decimales exige gestionar manualmente aritmética de punto fijo.

CKKS acepta un pequeño error de aproximación ($\sim 10^{-7}$) a cambio de poder codificar números reales de forma nativa. Eso lo convirtió en el esquema de referencia para *machine learning* y estadística sobre datos cifrados, donde ese error es irrelevante frente al ruido de los propios datos. No es adecuado cuando los resultados deben ser exactos.

TFHE: puertas lógicas en el toro. TFHE abandona por completo el modelo SIMD. Su base matemática es distinta: en lugar de explotar las raíces del polinomio $x^N + 1$ para crear slots independientes (como hace la familia RLWE), cifra cada bit como una posición en el *toro* $\mathbb{T} = \mathbb{R}/\mathbb{Z}$ — una estructura circular donde solo importa la parte decimal del número — y evalúa el cómputo como un circuito de puertas lógicas, una a una (Figura 5.14). Cada puerta ejecuta un bootstrapping automático, lo que elimina el concepto de «presupuesto de niveles»: el circuito puede ser arbitrariamente profundo sin planificación previa.

La fortaleza de TFHE es exactamente la debilidad de CKKS: comparaciones, condicionales y flujo de control. Un `if (x > y)` que en CKKS requiere aproximar la función signo con un polinomio de grado alto — consumiendo muchos niveles —, en TFHE se evalúa directamente bit a bit. La contrapartida: sin SIMD, sumar dos enteros de 32 bits exige encadenar cientos de puertas lógicas donde CKKS ejecutaría una sola operación. Las librerías modernas como Concrete (Zama) mitigan esto empaquetando enteros de n bits por cifrotexto y usando el *programmable bootstrapping* — que aplica

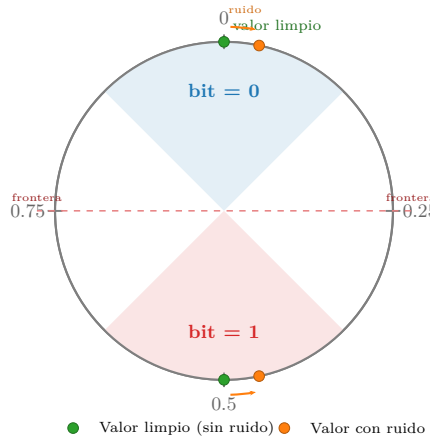


Figura 5.14: Codificación en el toro $\mathbb{T} = \mathbb{R}/\mathbb{Z}$. El bit 0 se codifica cerca de 0.0 (arriba) y el bit 1 cerca de 0.5 (abajo). El ruido desplaza la posición, pero el bit se recupera mientras no cruce la frontera de decisión.

cualquier función durante el proceso de limpieza del ruido, sin coste adicional — para evaluar funciones no lineales como ReLU o comparaciones en un solo paso.

Comparación. La Tabla 5.3 resume las diferencias entre los cuatro esquemas:

Tabla 5.3: Principales esquemas FHE: fundamentos, capacidades y compromisos.

Esquema	Base	Dato nativo	Fortaleza	Limitación principal
BFV	RLWE	Enteros exactos	Aritmética entera exacta, SIMD	Sin reales; bootstrap lento
BGV	RLWE	Enteros exactos	Como BFV, más eficiente a escala	Sin reales; bootstrap lento

Esquema	Base	Dato nativo	Fortaleza	Limitación principal
CKKS	RLWE	Reales aprox.	Punto flotante, SIMD	Error acumulado; inexacto
TFHE	LWE + RLWE	Bits / enteros pequeños	Comparaciones condicionales; LUTs	Sin SIMD; aritmética lenta

En nuestro ejemplo usamos CKKS porque necesitamos operar con números reales (salarios) y podemos tolerar un error de aproximación mínimo — exactamente el escenario para el que fue diseñado. Pero si el próximo encargo de Elena fuera identificar al paciente con la puntuación de riesgo más alta, comparar valores cifrados dejaría de ser un detalle menor, y Daniel tendría que evaluar TFHE en su lugar. En cambio, si el requisito de la incansable Elena fuera contar votos cifrados sin ningún margen de error, BFV o BGV serían los candidatos.

Preguntas de repaso

5.20. Pídele a un LLM: *«Explícame la diferencia entre cifrado homomórfico parcial, somewhat y fully homomorphic. Dame un ejemplo concreto de cada uno.»* ¿La respuesta es precisa?

5.21. Si `coeff_mod_bit_sizes=[60, 40, 40, 40, 60]`, ¿cuántas multiplicaciones encadenadas soporta el esquema?

5.22. BFV, BGV y CKKS comparten la misma base RLWE pero difieren en la codificación del mensaje. Describe qué codifica cada uno, su fortaleza principal y su limitación más importante. ¿En qué se distingue TFHE de los tres — tanto en fundamentos como en modelo de cómputo?

Lecturas Recomendadas

Han pasado varias semanas desde que Elena entró en la sala con su carpeta de post-its. El contrato con el proveedor se firmó, el análisis de riesgo corre cada noche sobre datos cifrados, y legal —para sorpresa de todos— dio su visto bueno sin pedir una sola reunión adicional. Daniel ha pasado de «jugar» con una tecnología desconocida a mantenerla en producción, y ha tenido que aprender por el camino más álgebra de retículos de la que esperaba necesitar en toda su vida. El viernes en que Elena le llamó desesperada porque «los números se desbordaban» le dejó claro que entender el *qué* no basta: hace falta el *por qué* para poder depurar y extender el sistema. Y Elena ya ha tanteado a Daniel su lista de deseos — comparar pacientes para priorizar el modelo de riesgo, algo que CKKS no resuelve bien y que empujará a Daniel a evaluar TFHE, o a asumir el coste de un segundo por operación con bootstrapping. Para quien quiera seguir este camino más allá de lo que cubre este capítulo, estas son las lecturas que lo hacen posible.

El mejor punto de partida para aprender en profundidad sobre cifrado homomórfico es el artículo original Gentry (2010), donde Gentry explica su propio avance en lenguaje accesible, sin demasiados formalismos; es ideal para entender *qué* resuelve el FHE y *por qué* funciona el bootstrapping. Como siguiente paso, Vaikuntanathan (2011) es una charla invitada en Foundations of Computer Science que profundiza en la mecánica del FHE sin requerir criptografía avanzada, explicando la evolución desde cifrado parcialmente homomórfico hasta FHE completo con claridad excepcional. Para una visión completa y actualizada del campo, Marcolla et al. (2022) ofrece una revisión exhaustiva que cubre todos los esquemas modernos (BGV, BFV, CKKS, TFHE), aceleración por hardware y despliegues reales.

Aunque el FHE garantiza la confidencialidad de los datos, no garantiza su integridad — el segundo pilar de la seguridad (confidencialidad, integridad y disponibilidad) —: no garantiza que el servidor haya calculado correctamente ni que el resultado no haya sido manipulado. La combinación natural

para añadir integridad son las pruebas de conocimiento cero (Goldwasser, Micali, y Rackoff 1989; Thaler 2022).

Sin embargo, el FHE protege los datos pero no el programa: el servidor conoce el programa que ejecuta. Para entender dónde encaja esta tecnología en el paisaje más amplio de la computación privada, Barak (2016) ofrece una introducción accesible escrita sobre cómputo seguro y ofuscación por uno de los investigadores más prominentes del campo. El artículo sitúa el FHE junto a otras dos primitivas complementarias. La primera es el *cómputo multiparte seguro* (MPC), donde varias partes computan conjuntamente sin que ninguna vea los datos de las demás; a diferencia del FHE, el MPC está ampliamente desplegado en producción hoy — con un overhead del orden de 10–100× sobre texto en claro — y tiene aplicaciones reales en detección de fraude entre bancos o análisis de datos genómicos entre instituciones. La segunda es la *ofuscación por indistinguibilidad* (iO), que permitiría cifrar también la lógica del programa — el escenario donde ni los datos ni el código son visibles para el servidor —, pero es hoy completamente impráctica (varios órdenes de magnitud más lenta que el propio FHE), razón por la que no tiene capítulo propio en este libro.

Referencias

Barak, Boaz. 2016. «Hopes, Fears, and Software Obfuscation». *Communications of the ACM* 59 (3): 88-96. https://doi.org/10.1145/2757276.

Gentry, Craig. 2009. «A Fully Homomorphic Encryption Scheme». Tesis doctoral, Stanford University. https://crypto.stanford.edu/craig/.

———. 2010. «Computing Arbitrary Functions of Encrypted Data». *Communications of the ACM* 53 (3): 97-105. https://doi.org/10.1145/1666420.1666444.

Goldwasser, Shafi, Silvio Micali, y Charles Rackoff. 1989. «The Knowledge Complexity of Interactive Proof Systems». *SIAM Journal on Computing* 18 (1): 186-208. https://doi.org/10.1137/0218012.

Marcolla, Chiara, Victor Sucasas, Marc Manzano, Riccardo Bassoli, Frank H. P. Fitzek, y Najwa Aaraj. 2022. «Survey on Fully Homomorphic Encryption, Theory, and Applications». *Proceedings of the IEEE* 110 (10): 1572-1609. https://doi.org/10.1109/JPROC.2022.3205665.

Rivest, Ronald L., Leonard Adleman, y Michael L. Dertouzos. 1978. «On Data Banks and Privacy Homomorphisms». *Foundations of Secure Computation*, 169-80.

Thaler, Justin. 2022. *Proofs, Arguments, and Zero-Knowledge*. Foundations y Trends en Privacy y Security. Now Publishers. https://doi.org/10.1561/3300000030.

Vaikuntanathan, Vinod. 2011. «Computing Blindfolded: New Developments in Fully Homomorphic Encryption». En *Proceedings of the 52nd Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 5-16. https://doi.org/10.1109/FOCS.2011.98.

Soluciones

2.1. BFV opera sobre enteros exactos (el descifrado recupera el valor exacto mediante redondeo) y es adecuado para consultas sobre bases de datos o conteos. CKKS opera sobre números reales aproximados (acepta un pequeño error de $\sim 10^{-6}$) y es el esquema de referencia para machine learning y estadística. Verifica que el LLM mencione que ambos comparten la base RLWE y que la diferencia clave está en la codificación del mensaje.

2.2. `poly_modulus_degree=16384` duplica la seguridad y permite más niveles multiplicativos, pero cada operación tarda más (los polinomios son el doble de grandes). Con `4096`, las operaciones son más rápidas pero se obtienen menos niveles multiplicativos y menor seguridad. El tiempo de ejecución escala aproximadamente con $O(n \log n)$ por operación NTT.

2.3. La fórmula $E[X^2] - (E[X])^2$ necesita una multiplicación element-wise (x_i^2) y una multiplicación del resultado de la media por sí mismo — dos multiplicaciones cifrotexto×cifrotexto. La alternativa $\frac{1}{n}\sum(x_i - \bar{x})^2$ requiere primero calcular $\bar{x}$ (sumas y rotaciones), luego restar a cada x_i (una suma por constante), y finalmente elevar al cuadrado cada diferencia — parece similar en multiplicaciones, pero exige primero materializar la media en cada slot (rotaciones extra). En CKKS, la primera fórmula es generalmente más eficiente: menos rotaciones y el mismo número de niveles multiplicativos.

2.4. (a) Versión directa — 2 eslabones. `media_x = ct_x.sum() * (1/n)` consume un eslabón, igual que `media_y = ct_y.sum() * (1/n)` en la rama paralela; `media_x * media_y` consume un segundo eslabón, dejando esa rama en el nivel 2. Por otro lado, `ct_x * ct_y` consume un eslabón y `.sum() * (1/n)` consume otro, dejando también esa rama en el nivel 2: ambas llegan al mismo eslabón, sin desajuste entre ellas, exactamente como en la versión ingenua de la varianza — y con el mismo riesgo de desbordamiento silencioso si el resultado no cabe en el margen de bits que queda en ese último eslabón. **(b) Versión optimizada — 1 eslabón.** Cifrando

también X/n e Y/n junto a X e Y (como hicimos con `ct_n` en la varianza), `media_x = ct_xn.sum()` y `media_y = ct_yn.sum()` no consumen ningún eslabón — son solo rotaciones —, así que `media_x * media_y` consume el único eslabón de esa rama; en la otra rama, `ct_x * ct_yn` ya incorpora la división por n en uno de los operandos y consume el único eslabón de la suya. Ambas ramas quedan en el nivel 1, con un eslabón de 40 bits de margen adicional respecto a la versión ingenua. **(c) ¿Cero eslabones?** No es posible: a diferencia de la Receta 3, donde los pesos eran públicos y la multiplicación por constante no consumía nivel, aquí X e Y son ambos privados, así que todo término cruzado — $X \cdot Y$ o $E[X] \cdot E[Y]$ — es necesariamente una multiplicación cifrotexto×cifrotexto. La covarianza es una función bilineal de dos cantidades cifradas, y ese único nivel es el mínimo teórico que ninguna reordenación del circuito puede eliminar.

2.5. Porque `features_cifrados * pesos` multiplica un cifrotexto por constantes en claro, lo que no consume niveles — es equivalente a escalar el polinomio interno. Si los pesos también estuvieran cifrados, sería una multiplicación cifrotexto × cifrotexto, que sí consume un nivel y requiere relinearización.

2.6. Con 10 000 features y $N_{\text{slot}} = 4096$, necesitas $\lceil 10\,000/4096 \rceil = 3$ cifrotextos para empaquetar todas las features. La predicción requiere 3 multiplicaciones por constantes (una por cifrotexto) y 3 sumas por rotación (una por cifrotexto), y finalmente sumar los 3 resultados escalares — una operación trivial sobre textos descifrados o una suma homomórfica adicional.

2.7. Se necesitan 2 niveles: uno para $x^2 = x \cdot x$ y otro para $x^3 = x^2 \cdot x$ (las multiplicaciones por constantes a, b, c, d y las sumas no consumen niveles). La configuración sería `coeff_mod_bit_sizes=[60, 40, 40, 60]` con `poly_modulus_degree=8192`.

2.8. El error explícito es una decisión de seguridad: en un sistema médico, un resultado «aproximado» silencioso podría causar un diagnóstico erróneo; en finanzas, una transacción con datos corruptos podría pasar desapercibida. El principio *fail-fast* garantiza que los errores se detecten en desarrollo, no en producción. Es preferible un sistema que se detiene a uno que produce resultados plausibles pero incorrectos.

2.9. Con profundidad 20: dimensionar 20 niveles requiere un `poly_modulus_degree` enorme (32768) para mantener la seguridad, lo que hace cada operación muy lenta. Con 5 niveles + bootstrapping cada 5 multiplicaciones, se necesitan 4 bootstrappings (~4 s extra) pero cada multiplicación individual es más rápida (polinomios más pequeños). Para circuitos profundos, el boots-

trapping periódico suele ser más eficiente que dimensionar estáticamente, especialmente cuando la profundidad no se conoce de antemano.

2.10. El bootstrapping evalúa homomórficamente el circuito de descifrado $(b - a \cdot s)$, lo que requiere cifrar la clave secreta s bajo sí misma. La *seguridad circular* es la hipótesis de que publicar $\mathrm{Enc}_s(s)$ no debilita el esquema. No está demostrada en general — es una suposición adicional que todos los esquemas FHE asumen. Verifica que el LLM distinga entre esta hipótesis y la seguridad estándar IND-CPA.

2.11. El servidor debe procesar *todos* los registros porque el vector de consulta está cifrado — no puede distinguir los ceros del uno. Si procesara solo un subconjunto, revelaría por eliminación que la posición consultada está en ese subconjunto, rompiendo la privacidad de la consulta.

2.12. Con $N_{\text{slot}} = N/2 = 4096$ slots por cifrotexto y 10 000 registros, el cliente necesita $\lceil 10\,000/4096 \rceil = 3$ cifrotextos (uno con el 1 en la posición correcta, los otros dos con solo ceros). El servidor debe procesar los 3 cifrotextos — no puede descartar ninguno sin filtrar información sobre la consulta.

2.13. Cada bloque adicional exige su propia multiplicación y suma sobre N_{slot} registros, así que el trabajo total del servidor es la suma del trabajo de todos los bloques — el mismo $O(n)$ de siempre, solo que repartido en más operaciones sobre cifrotextos más pequeños. Los sistemas de recuperación privada que particionan por categorías públicas (hospital, año, etc.) reducen el universo de registros candidatos a cambio de revelar esa categoría al servidor, asumiendo que no es información sensible.

3.1. Alternativas actuales insatisfactorias: (1) En sanidad se anonimiza o se recurre a acuerdos de confidencialidad, pero la anonimización es frágil (se pueden re-identificar pacientes cruzando variables) y los acuerdos no impiden que el proveedor vea datos en claro. (2) En finanzas se usan entornos controlados (*data clean rooms*) y acuerdos legales, pero el receptor sigue accediendo a los datos reales. (3) En ML multiorganización se centraliza la recopilación, exponiendo datos individuales. FHE eliminaría estas exposiciones al operar siempre sobre datos cifrados.

3.2. Empresas y librerías: Microsoft SEAL (C++, BFV/CKKS), IBM HELib (BGV/CKKS), Zama (Concrete, TFHE), Google (FHE transpiler para C++), Duality Technologies. El estado de madurez en 2024 es *producción limitada*: hay casos de uso reales (ML sobre datos cifrados, computación financiera) pero el overhead es aún elevado (10^4–$10^5\times$) para la mayoría de aplicaciones.

4.1. El coste es aceptable cuando el valor de la privacidad supera el coste computacional: datos genómicos, modelos financieros propietarios, votación electrónica, computación sobre datos médicos regulados. No es aceptable para aplicaciones de baja latencia (videojuegos, trading de alta frecuencia) o donde los datos no son sensibles.

4.2. Los avances principales entre 2015 y 2024 fueron: bootstrapping más rápido (TFHE genera evaluaciones de milisegundos), CKKS para aritmética aproximada (2017), compiladores FHE que transforman código normal a código homomórfico (Google, Zama), y aceleración hardware (Intel HEXL, proyectos DARPA DPRIVE). La brecha de rendimiento se redujo de $\sim 10^6\times$ a $\sim 10^4\times$ para muchas operaciones.

4.3. Los esquemas FHE basados en RLWE solo ofrecen dos operaciones nativas: suma y multiplicación de polinomios. Una comparación como `x > y` no es una operación aritmética — requiere evaluar una función no lineal (signo). Para implementarla, se aproxima la función signo mediante un polinomio de grado alto (típicamente grado 10-30), y cada multiplicación de ese polinomio consume un nivel del presupuesto. Un simple `if (x > y)` puede costar entre 4 y 10 niveles multiplicativos.

5.1. SVP pide encontrar el vector no nulo más corto del retículo; CVP pide encontrar el punto del retículo más cercano a un punto dado del espacio. CVP es al menos tan difícil como SVP porque, si se pudiera resolver CVP eficientemente, se podría usar para resolver SVP: basta con consultar CVP con el punto origen y descartar el vector nulo. La intuición de por qué SVP puede ser más fácil es que su punto de referencia (el origen) pertenece al propio retículo, lo que permite explotar simetrías internas de la estructura (invariancia por traslación, por ejemplo). En CVP, el punto objetivo es arbitrario y «rompe» esas simetrías — es un problema parametrizado por una entrada externa sin relación con la estructura periódica del retículo.

5.2. RSA se basa en la factorización de enteros, que el algoritmo de Shor resuelve en tiempo polinomial en un ordenador cuántico. Los problemas de retículos (SVP, CVP) no tienen un análogo cuántico eficiente conocido: los mejores algoritmos cuánticos siguen siendo exponenciales en la dimensión del retículo. Por eso los retículos son la base de la criptografía post-cuántica y, por extensión, del FHE.

5.3. En el anillo $R_q = \mathbb{Z}_q[x]/(x^N + 1)$, multiplicar por un polinomio $a(x)$ es equivalente a multiplicar por una matriz anticirculante determinada por los coeficientes de $a(x)$. Esto permite reemplazar la matriz aleatoria A de LWE (N^2 enteros, multiplicación $O(N^2)$) por un solo polinomio (N enteros, multiplicación $O(N \log N)$ vía NTT). Con $N = 8192$, las claves se reducen

~8 000× y las operaciones se aceleran ~1 000×.

5.4. La ecuación RLWE es $b = a \cdot s + e$, donde e es un error pequeño. Sin el error, recuperar s sería álgebra lineal trivial (dividir polinomios). Con él, b es un punto *cercano* al retículo generado por el anillo R_q, pero no exactamente en él — recuperar s equivale a encontrar el punto más cercano, que es CVP. Formalmente, Lyubashevsky, Peikert y Regev demostraron que resolver RLWE en el caso medio permite resolver SVP aproximado en retículos ideales en el caso peor. Verifica que el LLM mencione que la reducción aplica a retículos ideales (los que tienen la estructura de anillo R_q, no retículos generales) y que es el ruido e lo que convierte un problema trivial en uno intratable.

5.5. El producto sin plegar es $a(x) \cdot b(x) = 3x^5 + 8x^4 + 4x^3 + x^2 + 2x$. Aplicando $x^4 = -1$ (y por tanto $x^5 = -x$): $8x^4 \to -8$ y $3x^5 \to -3x$, lo que deja $4x^3 + x^2 - x - 8$. Reduciendo los coeficientes módulo 5 ($-1 \equiv 4$, $-8 \equiv 2$) se obtiene $4x^3 + x^2 + 4x + 2$.

5.6. Sin el polinomio aleatorio a, dos cifrotextos $c_1 = s + e_1 + m_1$ y $c_2 = s + e_2 + m_2$ permitirían al atacante calcular $c_1 - c_2 \approx m_1 - m_2$, revelando la diferencia entre los mensajes. El polinomio a, distinto en cada cifrado, actúa como máscara que hace cada cifrotexto indistinguible de ruido aleatorio.

5.7. En la suma homomórfica, los errores se suman: $e_{\text{sum}} = e_1 + e_2$. Si los errores iniciales son del orden de ϵ, tras k sumas el error es $k\epsilon$. En la multiplicación, los errores se multiplican con los mensajes: $e_{\text{mult}} \approx e_1 \cdot m_2 + e_2 \cdot m_1$, creciendo proporcionalmente al tamaño de los mensajes. Esto limita drásticamente el número de multiplicaciones encadenadas antes de que el ruido desborde el módulo.

5.8. La multiplicación homomórfica produce un triplete (d_0, d_1, d_2) que depende de s^2, en vez del par estándar (a, b) que depende solo de s. La relinearización usa una clave pública que cifra s^2 para convertir el triplete en un par estándar. Sin ella, cada multiplicación aumentaría el grado del cifrotexto en s, haciéndolo inmanejable tras pocas operaciones.

5.9. La lógica `if (m < 0) then m = -m` calcula el valor absoluto: $|m| = m \cdot \text{sign}(m)$. Como CKKS no soporta ramas condicionales, la función signo debe aproximarse mediante un polinomio (por ejemplo, la aproximación de Chebyshev de grado 15–27). La implementación evalúa ese polinomio sobre el cifrotexto — $\text{Enc}(\text{sign}(m)) \approx p(\text{Enc}(m))$ — y luego multiplica el resultado por $\text{Enc}(m)$: `enc_abs = enc_m * poly_sign(enc_m)`. Evaluar el polinomio consume $\lceil \log_2(\text{grado}) \rceil \approx 4$–5 niveles mediante el algoritmo de Paterson–Stockmeyer, más un nivel adicional para la multiplicación final: entre **5 y 10 niveles multiplicativos** en total, frente a una instrucción

en código convencional.

5.10. En CKKS, cada multiplicación duplica la escala del mensaje de Δ a Δ^2 y amplifica el error por un factor Δ. El rescaling divide todo el cifrotexto por Δ, restaurando la escala estándar y eliminando los 40 bits extra de error. Sin rescaling, la escala crecería exponencialmente y el módulo q se agotaría tras una sola multiplicación.

5.11. El circuito $a \cdot b + c \cdot d$ tiene dos multiplicaciones independientes (paralelas, no encadenadas) seguidas de una suma. La profundidad multiplicativa es 1. Basta con un solo nivel: `coeff_mod_bit_sizes=[60, 40, 60]` sería suficiente.

5.12. El bootstrapping evalúa el circuito de descifrado ($b - a \cdot s$) homomórficamente, lo que requiere operar con s dentro del dominio cifrado. Para ello se necesita un cifrado de s bajo la propia clave s. Esto requiere la hipótesis de *seguridad circular*: que cifrar la clave bajo sí misma no debilita el esquema. Todos los esquemas FHE actuales asumen esta propiedad, aunque no está demostrada en general.

5.13. Dimensionar los parámetros para evitar el bootstrapping requiere un módulo q proporcional al número de multiplicaciones, lo que exige un `poly_modulus_degree` cada vez mayor para mantener la seguridad. Para circuitos muy profundos, los polinomios serían tan grandes que cada operación sería más lenta que hacer bootstrapping periódico. El bootstrapping es inevitable cuando la profundidad del circuito no se conoce de antemano o es demasiado grande para dimensionar estáticamente.

5.14. El polinomio $x^N + 1$ tiene N raíces complejas, pero como los coeficientes del polinomio mensaje son reales, las evaluaciones en raíces conjugadas son también conjugadas: $m(\bar{\omega}) = \overline{m(\omega)}$. Las raíces se agrupan en $N/2$ pares conjugados, y cada par aporta un solo valor independiente. Por eso hay $N/2$ slots, no N.

5.15. Los 4091 slots restantes contienen ceros. El coste de las operaciones no se ve afectado: una suma o multiplicación homomórfica procesa *todos* los $N/2 = 4096$ slots en paralelo, estén o no ocupados. El coste es por cifrotexto, no por dato — los slots vacíos simplemente suman o multiplican ceros sin coste adicional.

5.16. Los slots no coinciden con los coeficientes del polinomio: están codificados mediante una variante de la DFT (la *canonical embedding*). El descifrado homomórfico del bootstrapping produce el polinomio en su representación de coeficientes, pero la reducción modular debe aplicarse a cada slot por separado. *CoeffsToSlots* convierte de coeficientes a slots me-

diante una multiplicación matricial homomórfica, lo que consume varios niveles multiplicativos adicionales — niveles que se restan del presupuesto recién restaurado.

5.17. El árbol de sumas necesita $\log_2(4096) = 12$ rotaciones y 12 sumas homomórficas. En cada paso se rota por una potencia de 2 $(1, 2, 4, \ldots, 2048)$ y se suma el resultado con el vector original, de modo que tras 12 pasos cada slot contiene la suma de todos los elementos.

5.18. La sustitución $x \to x^k$ que implementa la rotación transforma el cifrotexto $(a(x), b(x))$ en $(a(x^k), b(x^k))$, que queda cifrado bajo la clave $s(x^k)$ en lugar de $s(x)$. Para poder descifrar con la clave original, se necesita un *key-switching* que requiere material precalculado — la clave de Galois — específico para cada valor de k. Distintas cantidades de rotación corresponden a distintos exponentes k, y cada uno necesita su propia clave.

5.19. En un registro SIMD de CPU, un shift circular reordena las posiciones del registro sin coste criptográfico — es una operación directa sobre bits. En FHE, la rotación logra el mismo efecto lógico (desplazar posiciones del vector), pero requiere un automorfismo sobre el polinomio seguido de key-switching, lo que añade ruido y necesita una clave de Galois precalculada. La similitud es funcional: ambos mueven datos dentro de un «registro ancho». La diferencia es el coste: nanosegundos en CPU, milisegundos en FHE, más la necesidad de claves adicionales.

5.20. Parcial: soporta UN SOLO tipo de operación sin límite (RSA→multiplicaciones, Paillier→sumas). Somewhat (SHE): soporta sumas Y multiplicaciones pero con un número limitado de operaciones antes de que el ruido imposibilite el descifrado. Fully (FHE): soporta sumas y multiplicaciones sin límite, gracias al bootstrapping que resetea el ruido periódicamente. Verifica que el LLM no confunda SHE con FHE.

5.21. Los niveles intermedios (`40, 40, 40`) consumen un nivel de profundidad multiplicativa cada uno. Con tres niveles intermedios de 40 bits, el esquema soporta **3 multiplicaciones encadenadas**. Los niveles extremos (`60`) se reservan para la escala (encoding) y el descifrado.

5.22. *Codificación*: BFV escala el mensaje entero por $\lfloor q/t \rfloor$ y recupera el resultado exacto por redondeo; BGV hace lo mismo con gestión inversa de módulos; CKKS escala por Δ y acepta un error de aproximación para representar reales. *Fortalezas*: BFV/BGV garantizan aritmética entera exacta; CKKS permite punto flotante y es natural para ML. *Limitaciones*: BFV/BGV no representan reales; CKKS acumula error. TFHE se distingue en ambos ejes: usa LWE (no RLWE) — vectores de enteros en lugar de polinomios — y codifica un solo bit por cifrotexto en un toro. Su modelo

de cómputo es por puertas lógicas con bootstrapping automático tras cada puerta (~10 ms), lo que lo hace ideal para comparaciones y condicionales pero ineficiente para aritmética masiva (sin paralelismo SIMD).